



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

Факултет Приложна математика и информатика



ДИПЛОМНА РАБОТА

Тема : Приложение за управление на лична здравна
информация

Дипломант : Таня Георгиева Узунова

Фак. №: 961324004

Специалност : Анализ на големи масиви и потоци данни

Образователно-квалификационна степен : Магистър

Дипломен ръководител :доц. Д-р Златко Захариев

ТУ-СОФИЯ 2025

Въведение.....	4
Глава 1.Теоретични основи и преглед на съществуващи решения	5
1.1.Актуалност на темата	5
1.2.Проблеми от гледна точка на пациента.....	6
1.3.Основни цели на проекта MedJ	7
1.4.Дигитални медицински дневници.....	8
1.5.Етични и правни аспекти	10
Глава 2.Технологии и среди за разработка.....	11
2.1Използвана програмна среда:	11
2.1.1Backend: Python и Django	11
2.1.2Frontend - Класически подход със съвременни инструменти	13
2.1.3Среда за виртуализация: Docker и Docker Compose.....	14
2.1.4Docker Compose.....	15
2.2Интелигентна обработка: OCR, GPT и системи за тагове	15
2.2.1Бази данни: PostgreSQL	17
2.3Библиотеки и зависимости.....	17
2.4Инструменти за разработка.....	18
Глава 3.Анализ на изискванията към проекта.....	19
3.1Методология на проучването	19
3.1.1Целева аудитория на проучването:	19
3.2Инструмент и разпространение:.....	20
3.3Анализ на резултатите от анкетата:	20
3.4Психологически аспект (UX - User Experience).....	23
3.5Намаляване на когнитивното натоварване:	24
3.6Създаване на усещане за овластяване (Empowerment):	24
3.7Определяне на основните функционалности.....	25
3.8Извличане на текст чрез Google Cloud Vision API	25
3.9Избор на фокус	26
Глава 4.Архитектура на системата.....	28
4.1.1Поток на данните и взаимодействие между компонентите ...	30

4.2Сервизният модул изпраща файла към Google Cloud Vision API за OCR.....	30
4.3Структура на Django проекта.....	31
4.4Основни модели и техните връзки.....	32
4.4.1MedicalEvent (Медицинско събитие).....	33
4.4.2MedicalDocument (Медицински документ).....	34
4.4.3Tag (Етикет) и Specialty (Специалност).....	34
4.5Seed данни.....	35
Глава 5.Техническа реализация, приложение и бъдещо развитие.....	36
5.1Целева аудитория и крайни потребители.....	36
5.1.1Пациенти (Основна потребителска група).....	36
5.1.2Лекари и медицински специалисти (Вторична потребителска група).....	37
5.2Техническа реализация на ключови функционалности.....	38
5.2.1Процес по качване и интелигентна обработка на документи	38
5.2.2. Система за сигурно споделяне на данни	40
5.3Потенциал и перспективи за бъдещо развитие.....	42
5.3.1Разширения на функционалността.....	42
5.3.2Технологични и архитектурни подобрения	43
Заклучение.....	44
Цитати.....	Er
	ror! Bookmark not defined.
Приложения:.....	50

Въведение

Дигиталната еволюция на здравеопазването е необратим и фундаментален процес, който прекроява начина, по който медицинските услуги се предоставят, управляват и консумират в глобален мащаб^[1].

Управлението на личната здравна информация на индивидуално ниво остава парадоксално архаично и фрагментирано. Пациентите продължават да бъдат пасивни носители на десетки, а понякога и стотици, документи като епикризи, лабораторни изследвания, рецепти, амбулаторни листове и образни изследвания, които в по-голямата си част съществуват единствено в хартиен вид. Съгласно чл. 29, ал. 1 от посочената наредба, лечебните заведения съхраняват медицинската документация за извършените от тях прегледи и изследвания три години след извършването им. В ал. 3 на чл. 29 от същия нормативен акт е предвидено, че след изтичане на срока по ал. 1 документацията на хартиен носител се предоставя на пациента, а при отказ или невъзможност от него да я приеме, подлежи на унищожаване^[2].

Този традиционен, аналогов подход към съхранението на медицинска документация е обременен от множество системни проблеми. Рискът от безвъзвратна загуба или физическа повреда на важна информация е постоянен. Достъпът до данни при спешни случаи или при пътуване е силно затруднен, а понякога и невъзможен. Липсва механизъм за проследяване на ключови здравни показатели в реално време, което лишава както пациента, така и лекаря от ценна информация за развитието на дадено състояние.

Тази фрагментация често води до забавяне на диагностичния процес, назначаване на дублиращи се и ненужни изследвания и в крайна сметка – до неоптимално лечение и повишени разходи както за здравната система така и за пациента. Липсата на централизирана и лесно достъпна лична медицинска история създава сериозни затруднения.

Крайната цел на проекта е да предостави инструмент, който овластява пациентите, като им дава възможност да бъдат начело в управлението на собствената си здравна медицинска история, да подобрят комуникацията с медицинските специалисти, да повишат качеството на грижа което биха получили и да вземат по-информирани решения за своето лечение, здраве и време.

Глава 1. Теоретични основи и преглед на съществуващи решения

1.1 Актуалност на темата

В последните години дигиталната трансформация се превърна в основен двигател за модернизация в почти всеки сектор на обществения живот, като здравеопазването не прави изключение. Необходимостта от бърз, сигурен и ефективен достъп до медицинска информация е по-осезаема от всякога. В България този процес намери своя катализатор в лицето на Националната здравноинформационна система (НЗИС), по-известна като „еЗдраве“. Пускането ѝ в експлоатация през 2020 г. бележи ключов момент в дигитализацията на българското здравеопазване, като до средата на 2024 г. системата е обработила над 345 милиона електронни здравни записи, включително над 54 милиона електронни рецепти и 53 милиона електронни направления.

Тези впечатляващи статистики демонстрират не само технологичния капацитет на системата, но и готовността на медицинските специалисти и пациентите да приемат и използват дигитални инструменти в своето ежедневие.

Въпреки този неоспорим напредък, съществуващата парадигма на дигитализацията оставя един фундаментален проблем неразрешен – пасивната роля на пациента в управлението на собствената му здравна информация. Системата „еЗдраве“ е проектирана като институционална, „отгоре-надолу“ платформа. В нея данните се въвеждат и управляват единствено от медицински специалисти и здравни институции, които са сключили договор с Националната здравноосигурителна каса (НЗОК).^[3, 4] Пациентът има достъп до своето електронно здравно досие, но този достъп е предимно с информативна цел – той може да преглежда, но не и да добавя, редактира или управлява информацията в него.^[5] Тази архитектура създава критичен „пропуск в контрола“, който ограничава възможността на пациента да изгради пълна, цялостна и хронологична картина на своето здравословно състояние.

Същевременно, традиционното съхранение на медицински документи в хартиен вид продължава да бъде широко разпространена практика, която носи със себе си редица проблеми. Хартиените документи са уязвими на загуба, повреда или избледняване с времето. Тяхното споделяне между различни лекари и клиники е тромаво и неефективно, а извличането на информация за проследяване на тенденции в здравните показатели е практически невъзможно без ръчна обработка или специализирани знания.

1.2 Проблеми от гледна точка на пациента

Медицинската история на един човек често е фрагментирана и разпръсната между личния лекар, различни специалисти, лаборатории, болници и дори държави. Тази разпокъсаност принуждава пациента да бъде „жив носител“ на своята документация, което е свързано с постоянен риск от загуба на важни документи и история. Тези проблеми са особено ясно изразени при няколко специфични потребителски групи:

•Хронично болни пациенти:

За хората със заболявания като диабет, хипертония или автоимунни състояния, постоянното проследяване на ключови показатели е от жизненоважно значение. Те се нуждаят от инструмент, който им позволява не само да съхраняват резултатите от кръвни изследвания, но и да визуализират промените в тях във времето. Споделянето на пълна и добре организирана дългосрочна история с ендокринолог, кардиолог или друг специалист може значително да подобри качеството на диагностиката и лечението.

•Родители:

Управлението на здравето на децата е сложен процес, включващ проследяване на имунизационен календар, редовни профилактични прегледи, както и документиране на всички прекарани заболявания и предписани лечения. Наличието на дигитален дневник, в който цялата тази информация е събрана на едно място, би улеснило значително родителите и би осигурило приемственост в грижата, дори при смяна на педиатъра.

•Придружители на тежкоболни или възрастни пациенти:

Хората, които се грижат за свои близки с тежки или хронични заболявания, често поемат ролята на посредник между пациента и здравната система. За тях е изключително трудно да поддържат ред в огромно количество медицинска документация и да предоставят адекватна информация на лекуващите лекари. Инструмент, който позволява управлението на профила на друг човек, би бил неоценима помощ в тази отговорна задача.

1.3 Основни цели на проекта MedJ

„MedJ“ – персонален здравен дневник, поставя пациента в центъра и му предоставя пълен контрол върху неговата медицинска информация.

Основните цели на проекта са формулирани, както следва:

- **Създаване на сигурна, интуитивна и централизирана платформа:**

Да се разработи уеб приложение, което позволява на потребителите да съхраняват, организират и управляват цялата си медицинска история на едно място, независимо от времето през което са били получени или източника на документите (държавни, български или чуждестранни лечебни заведения, клиники, лаборатории, частни кабинети или дори онлайн услуги).

- **Разработване на система за лесна дигитализация на документи:**

Да се имплементира функционалност за качване на сканирани или заснети медицински документи (епикризи, резултати от изследвания, направления, ТЕЛК решения, оперативни протоколи, документи за пътуване, сертификати, разчитане на ЯМР, Ехография или ултразвук, административни документи, медицински становища) и автоматичното им преобразуване в текстов формат чрез технология за оптично разпознаване на символи Optical Character Recognition (OCR).

- **Интегриране на изкуствен интелект за структуриране на данни:**

Да се използва силата на големите езикови модели Artificial intelligence (AI/GPT) за автоматично извличане, анализиране и структуриране на ключова информация от текстовото съдържание на документите. Целта е превръщането на неструктуриран текст в полезни, машинночетими данни (например, показатели от кръвни изследвания с техните стойности и референтни граници).

- **Осигуряване на инструменти за визуализация и анализ:**

Да се предоставят на потребителя функционалности за визуализация на структурираните данни под формата на графики и таблици, което улеснява проследяването на здравни показатели във времето.

- **Имплементиране на гъвкав и сигурен механизъм за споделяне:**

Да се създаде система, която позволява на потребителите да споделят избрана част от своята медицинска история с лекари по сигурен и контролиран начин – чрез временни, защитени с уникален токен линкове или QR кодове, без да се изисква регистрация от страна на лекаря.

Крайната цел на проекта MedJ е да предостави инструмент, който овластява пациентите, като им дава възможност да бъдат активни участници в управлението на собственото си здраве, да подобрят комуникацията с медицинските специалисти и да вземат по-информирани решения за своето лечение и превенция.

1.4 Дигитални медицински дневници

Пазарът на дигитални здравни приложения предлага разнообразие от решения, които обаче могат да бъдат условно разделени на няколко основни категории според техния фокус и целева аудитория. За целите на настоящия анализ ще разгледаме три ключови примера, които очертават съществуващия технологичен пейзаж: глобалните платформи за следене на жизнени показатели по време на фитнес като **“Google Fit”** и **“Apple Health”**, и националната здравноинформационна система на България – **„еЗдраве“**.

•“Google Fit” и “Apple Health”

Водещи платформи в сферата на потребителското здраве, но техният основен фокус е върху проследяването на физическата активност и общи показатели за благосъстояние (wellness). Тези приложения са изключително добри в събирането на данни от сензори на смартфони и смарт часовници – брой крачки, изминато разстояние, сърдечен ритъм, качество на съня и др.^[6, 7]

Те предлагат отлична интеграция с широк набор от устройства и приложения на трети страни, създавайки богата екосистема за потребителя. Apple Health, предлага и възможност за съхранение на някои видове клинични данни чрез стандарта “HealthKit”, но процесът по импортиране на документи като епикризи или лабораторни изследвания не е основна функционалност и остава предизвикателство за крайния потребител.^[8–10] Основното ограничение на тези платформи е, че те са проектирани предимно за данни, генерирани от потребителя или от носими устройства, а не за обработка и структуриране на сложни клинични документи, издадени от здравни институции.

•Националната здравноинформационна система „еЗдраве“

В България представлява другата крайност на спектъра. Това е централизирана, институционална система, чиято основна цел е да осигури единен формат и стандарт за обмен на медицинска информация между лекари, болници, лаборатории и аптеки. Нейните предимства са неоспорими: национално покритие, стандартизация на данните и директна интеграция със здравноосигурителната система.

Основното ограничение на системата, както беше споменато, е липсата на контрол от страна на пациента. Потребителят не може да добавя документи от частни прегледи, консултации в чужбина или да въвежда лични бележки и наблюдения. Системата отразява официалната, но не непременно пълната здравна история на пациента.^[11] Една от най-големите слабости на системата е самата тя, поради голямото количество информация, огромния брой пациенти в здравната система, много лични лекари дори да получат от пациента документи от източници различни от държавните, лекарите почти никога нямат време тази информация да я въведат в електронната система, което прекъсва връзката между НЗОК и частните медицински лица.

Проектът “MedJ” е проектиран да заеме нишата между тези два типа системи. Той не се конкурира с фитнес тракерите, нито цели да замени националната здравна система. Вместо това, предоставя на пациента липсващия инструмент – лично и контролирано от него пространство, където може да обедини всички свои медицински документи, независимо от техния произход, и да ги превърне в структурирана, лесна за използване споделяне без ограничения информация.

Характеристика	Apple Health / Google Fit	еЗдраве (НЗИС)	MedJ
Основен източник на данни	Генерирани от потребителя (фитнес, уелнес)	Клинични данни от здравни институции	Всички видове медицински документи, качени от потребителя
Контрол върху данните	Потребител	Институция / Лекар	Потребител
Качване и OCR на документи	Ограничено / Липсва	Не се поддържа от пациента	Основна функционалност
AI структуриране на данни	Липсва	Липсва	Основна функционалност
Визуализация на клинични данни	Ограничена до специфични показатели	Базова визуализация на резултати	Фокус върху графики и тенденции
Споделяне с лекари	Ограничено	Достъп за регистрирани специалисти	Гъвкаво, временно и сигурно споделяне чрез линк/QR

Фигура 1. Сравнителен анализ на съществуващи здравни платформи

1.5 Етични и правни аспекти

Разработването на приложение, което съхранява и обработва здравна информация, е свързано с огромна отговорност по отношение на защитата на личните данни и сигурността. Здравните данни се класифицират като „специална категория лични данни“ съгласно Общия регламент за защита на данните (GDPR) на Европейския съюз, което налага най-строги изисквания за тяхната обработка^[11, 12].

В България, освен GDPR, се прилага и Законът за защита на личните данни (ЗЗЛД), който допълва и конкретизира европейската рамка.

При проектирането на MedJ са взети предвид няколко ключови принципа на GDPR:

- **Законосъобразност, добросъвестност и прозрачност:**

Обработката на данни в MedJ се извършва единствено и само въз основа на изричното и информирано съгласие на потребителя, дадено при регистрация.

- **Ограничение на целите:**

Данните се събират с единствената цел да служат на потребителя за управление на неговия личен здравен дневник и не се използват за никакви други цели (напр. маркетинг, продажба на трети страни).

- **Свеждане на данните до минимум (Data Minimisation):**

Системата събира само данните, които са абсолютно необходими за нейната функционалност.

Изграждането на доверие у потребителите е от първостепенно значение. Потребителите трябва да са сигурни, че тяхната най-чувствителна информация е в безопасност. Поради тази причина, архитектурата на MedJ е проектирана с фокус върху контрола от страна на потребителя. Функцията за споделяне е пример за прилагане на принципа „Privacy by Design“. Вместо да предоставя постоянен достъп, системата генерира временни, уникални линкове с ограничен срок на валидност, като потребителят сам избира кои точно документи да сподели. Това гарантира, че достъпът е ограничен до конкретната цел (напр. консултация с лекар) и се прекратява автоматично след това.

Глава 2. Технологии и среди за разработка

Изграждането на стабилна, сигурна и мащабируема уеб платформа изисква внимателен и обоснован подбор на технологна рамка, която да отговаря на специфичните изисквания на проекта. Решението за избор на конкретни технологии не е произволно, а е от необходимостта за бърза и точна обработка на данни, предоставяне на интуитивен и отзивчив потребителски интерфейс, както и осигуряване на ефективен и предвидим процес на разработка, тестване и поддръжка.

Целта беше съчетанието на стабилността при разработка на класическо, монолитно уеб приложение с гъвкавостта и мощта на специализирани външни и вътрешни микроуслуги. Този хибриден подход позволява основната работна логика, управлението на потребителите и данните да бъдат централизирани в ядро, изградено с Django, докато ресурсоемките и специализирани задачи като оптичното разпознаване на символи и анализът с изкуствен интелект се делегират на отделни, независимо работещи компоненти. Това осигурява оптимален баланс между производителност, сигурност и възможност за бъдещо разширение.

2.1 Използвана програмна среда:

2.1.1 Backend: Python и Django

Основата на сървърната част (backend) на приложението е изградена върху езика за програмиране Python^[13] и уеб рамката Django^[14]. Тази комбинация е стратегически избрана, за да отговори на комплексните изисквания на проекта.

2.1.1.1 Python

Ключово предимство е огромната екосистема от библиотеки, особено в сферата на обработката на данни, научните изчисления и изкуствения интелект.

Библиотеки като 'Pillow' за обработка на изображения, 'ReportLab' за генериране на PDF документи и 'requests' за комуникация с външни API са само част от инструментите, които са директно използвани в MedJ и са достъпни благодарение на Python.

2.1.1.2 Django

Уеб рамка от високо ниво, която следва архитектурния модел MVT (Model-View-Template) и насърчава бързата разработка на сигурни и лесни за поддръжка уеб приложения. Изборът на Django е обоснован от неговата

философия "Всичко включено" (Batteries-Included), която предоставя на разработчика огромен набор от вградени, тествани и надеждни компоненти.

•ORM (Object-Relational Mapper):

Вграденият ORM на Django е един от най-мощните му инструменти. Той позволява на разработчиците да дефинират структурата на базата данни чрез Python класове (модели), както е видно от файла ``records/models.py`` [вж. Приложение 1]. Взаимодействието с базата данни (създаване, четене, филтриране, изтриване на записи) се осъществява чрез извикване на Python методи върху тези обекти, което елиминира нуждата от писане на ръчни SQL заявки. Това не само прави кода по-четим и сигурен (тъй като ORM-ът се грижи за екранирането на заявките и предотвратяването на SQL Injection атаки), но и осигурява преносимост между различни типове бази данни.

•Сигурност:

Django предоставя вградена защита срещу редица често срещани уеб уязвимости. Механизмите за защита от Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), Clickjacking и сигурното управление на пароли (чрез хеширане) са активни по подразбиране. За приложение като MedJ, което борави с най-чувствителния тип лични данни, тези вградени гаранции за сигурност са от първостепенно значение.

•Автентикация и управление на потребители:

Django идва с цялостна система за автентикация, която управлява потребителските акаунти, групи и права. Тази система е използвана като основа в MedJ, като е разширена с модела ``Profile`` за добавяне на специфични за приложението полета.

•Административен панел:

Една от най-отличителните черти на Django е неговият автоматично генериран административен интерфейс. С няколко реда код в ``records/admin.py`` [вж. Приложение 2], разработчиците получават мощен уеб-базиран интерфейс за управление на всички данни в базата данни, което е безценно по време на разработка, тестване и поддръжка.

•Модулност и организация:

Проектът следва стандартната за Django структура, като цялата основна функционалност е капсулирана в приложението ``records``. Това насърчава ясното разделение на отговорностите и прави кода по-лесен за разбиране и разширяване в бъдеще.

2.1.2 Frontend - Класически подход със съвременни инструменти

Потребителският интерфейс (frontend) е реализиран чрез класическия подход на рендиране на шаблони от страна на сървъра (Server-Side Rendering - SSR), като са използвани стандартни и доказани уеб технологии: **HTML**^[15], **Tailwind**^[16] и **JavaScript**^[17]. Този подход е избран пред алтернативата на Single-Page Application (SPA) с цел опростяване на разработката, по-бързо първоначално зареждане на страниците и по-лесна индексация от търсачки за публичната част на сайта.

2.1.2.1 HTML

Използва за семантичното структуриране на съдържанието. Използването на семантични тагове като `<nav>`, `<main>`, `<section>` и `<article>` в шаблоните `records/templates/` не само подобрява достъпността за асистирани технологии, но и прави кода по-разбираем както за разработчиците, така и за търсачките.

2.1.2.2 Tailwind CSS

Представява "utility-first" CSS рамка, която революционизира начина на стилизиране. Вместо да се пишат отделни CSS файлове с специфично създадени класове, стиловете се прилагат директно в HTML чрез комбинация от малки класове (напр. `flex`, `pt-4`, `text-center`, `bg-blue-500`). Този подход, макар и необичаен в началото, носи огромни предимства за проект като MedJ:

- **Консистентност на дизайна**

Всички стилове се базират на предварително конфигурирана дизайн система (цветова палитра, размери, разстояния), което гарантира, че целият интерфейс изглежда унифициран и професионален.

- **Бързина на разработка**

Елиминира се нуждата от постоянно превключване между HTML и CSS файлове и измисляне на имена за класове.

- **Отзивчив дизайн**

Tailwind има интуитивни префикси за прилагане на стилове при различни размери на екрана (напр. `md:flex`, `lg:text-lg`), което прави създаването на напълно отзивчив интерфейс, работещ отлично на всякакви устройства, изключително лесно.

•Оптимизация на производителността

При компилация, Tailwind сканира всички файлове и генерира CSS файл, който съдържа само използваните класове, което води до минимален размер и по-бързо зареждане.

2.1.2.3 JavaScript

Използван е целенасочено за подобряване на интерактивността, без да се усложнява архитектурата. Неговата роля в MedJ е да добави динамично поведение към генерираните от сървъра страници. Както е видно от файловете в ``theme/static/js/`` [вж. Приложение 3], JavaScript се използва за:

•Асинхронни заявки (AJAX):

Файлове като ``upload_logic.js`` и ``filters.js`` използват Fetch API за комуникация със сървъра във фонов режим. Например, при качване на документ, файлът се изпраща към бекенда, без страницата да се презарежда, а интерфейсьт се обновява динамично, за да покаже статуса на обработка.

•Манипулация на DOM:

Управление на видимостта на елементи, модални прозорци, табове и други интерактивни компоненти, често с помощта на малки библиотеки като Flowbite^[18].

•Валидация на форми от страна на клиента:

Осигурява бърза обратна връзка на потребителя при попълване на форми, преди данните да бъдат изпратени към сървъра.

2.1.3 Среда за виртуализация: Docker и Docker Compose

За да се постигне консистентна, изолирана и лесно преносима среда за разработка и бъдещ “deployment”, проектът MedJ разчита изцяло на технологии за виртуализация на ниво операционна система **Docker**^[19] и **Docker Compose**^[20]. Този подход решава класическия проблем в софтуерната разработка "на моята машина работи", като гарантира, че приложението се изпълнява в идентична среда, независимо от хост системата.

Docker позволява "опаковането" на всяка част от приложението в леки, стандартизирани единици, наречени “контейнери”. Всеки контейнер включва кода на приложението и абсолютно всички негови зависимости (библиотеки, системни инструменти, изпълнима среда). В проекта MedJ са дефинирани няколко Docker образа:

- **`web`**

Базиран на официален Python образ, **`docker/web/Dockerfile`** [вж. Приложение 4] инсталира всички необходими Python пакети от **`requirements.txt`** [вж. Приложение 5] и конфигурира средата за изпълнение на Django приложението.

- **`ocrapi`**

Подобен образ, дефиниран в **`docker/ocrapi/Dockerfile`** [вж. Приложение 6], който инсталира зависимостите, необходими за OCR микроуслугата от **`ocrapi/requirements-ocr.txt`** [вж. Приложение 7].

2.1.4 Docker Compose

Инструмент, който оркестрира тези контейнери и ги кара да работят заедно като едно цялостно приложение. Файлът **`docker/compose/docker-compose.dev.yml`** дефинира всички услуги, необходими за стартиране на системата в режим на разработка:

- **`db` услуга:**

Стартира контейнер с PostgreSQL^[21], като използва официален образ от Docker Hub. Данните се съхраняват в Docker volume, за да не се губят при рестартиране на контейнера.

- **`web` услуга:**

Изгражда и стартира контейнера за Django приложението. Свързва го с **`db`** услугата, така че Django да може да комуникира с базата данни. Монтира (**`volumes`**) основната директория с кода на проекта вътре в контейнера, което позволява на разработчика да прави промени в кода и те да се отразяват незабавно, без да е необходимо ново изграждане на образа.

- **`ocrapi` услуга:**

Стартира контейнера за OCR микросървиса, който работи независимо и комуникира с **`web`** услугата по мрежата, дефинирана от Docker Compose.

С една единствена команда (**`docker-compose up`**) разработчикът може да стартира цялата сложна система, включително база данни и микроуслуги, което драстично опростява настройката на работната среда и гарантира, че всички членове на екипа работят в идентични условия.

2.2 Интелигентна обработка: OCR, GPT и системи за тагове

Това е технологичното ядро, което отличава MedJ от обикновено хранилище за файлове. То превръща неструктурираните данни от документи в полезна, систематизирана и лесна за анализ информация.

•Optical Character Recognition (OCR) технологии:

Първата стъпка в този процес е преобразуването на изображение (сканиран документ или снимка) в суров, машиночитаем текст. За тази цел е избрана услугата **Google Cloud Vision API**^[22]. Причините за този избор са:

•Изключителна точност:

Моделите на Google са обучени върху огромен набор от данни и предлагат една от най-високите точности на пазара, особено при разпознаване на текст на български език.

•Справяне с трудни условия:

Услугата се справя добре с документи с неидеално качество – леко замъглени, под ъгъл или с ръкописни бележки, което е често срещано при медицинската документация.

•Архитектурна имплементация:

Както беше споменато, OCR процесът е изнесен в отделен микросървис (`ocrapi`). Това е ключово архитектурно решение, защото OCR обработката може да отнеме няколко секунди. Като се изпълнява асинхронно в отделна услуга, тя не блокира основния Django уеб сървър и потребителският интерфейс остава отзивчив.

•Изкуствен интелект (GPT) за анализ:

Извлеченият от OCR текст е просто низ от символи. За да се извлече смисъл от него, се използва мощта на големите езикови модели (LLM), в случая **OpenAI GPT**^[23]. Интеграцията се осъществява чрез `GPTClient` (`records/views/api_upload.py`) [вж. Приложение 8]. Ролята на GPT е да извърши семантичен анализ на текста чрез "промпт" – подаване на текста към модела заедно с прецизно формулирани инструкции.

Например, промптът за обработка на епикриза инструктира модела: "Ти си медицински асистент. От следния текст извлечи диагноза, име на лекуващ лекар, дата на издаване и кратко обобщение от 3-4 изречения. Върни резултата в JSON формат със следните ключове: 'diagnosis', 'practitioner_name', 'event_date', 'summary'". Моделът разбира тези инструкции и трансформира неструктурирания текст в структуриран JSON обект, който може лесно да бъде записан в базата данни и показан в потребителския интерфейс[вж. Приложение 9].

- Системи за тагове и филтрация:** След като информацията е структурирана, тя трябва да бъде лесно откриваема. MedJ имплементира гъвкава система за категоризация чрез тагове. Тази система е реализирана директно чрез възможностите на Django ORM. Моделът `Tag`, дефиниран в `records/models.py`, има връзка (`ManyToManyField`) с основния модел

``MedicalEvent``. Това позволява всяко медицинско събитие да бъде асоциирано с множество тагове (напр. "Епикриза", "Кардиология", "Болница Токуда"). Когато потребителят използва филтрите в интерфейса, JavaScript код (``theme/static/js/filters.js``) изпраща AJAX заявка към Django. Бекендът (``records/views/events.py``[вж. Приложение 10]) приема тези тагове като параметри и конструира динамична ORM заявка (``MedicalEvent.objects.filter(tags__name__in=[...])``), която ефективно филтрира и връща само релевантните резултати, осигурявайки бързо и интерактивно търсене.

2.2.1 Бази данни: PostgreSQL

Изборът на система за управление на бази данни (СУБД) е критичен за надеждността и интегритета на данните, особено когато става въпрос за медицинска информация. За проекта MedJ е избрана **PostgreSQL**^[24] версия 16, обектно-релационна СУБД с отворен код. Аргументите в полза на **PostgreSQL** са многобройни:

- **Надеждност и транзакционна сигурност:** **PostgreSQL** е известна със своята стриктна поддръжка на ACID свойствата (Atomicity, Consistency, Isolation, Durability), което гарантира, че транзакциите се изпълняват надеждно и данните остават в консистентно състояние дори при системни сринове.
- **Разширени типове данни:** **PostgreSQL** поддържа широк набор от типове данни, включително JSONB, който позволява ефективното съхранение и индексирание на неструктурирани или полуструктурирани JSON[25] данни. Това е изключително полезно за съхраняване на резултатите от AI анализа, които често са в JSON формат.
- **Мащабируемост и производителност:** Системата е проектирана да се справя с големи обеми от данни и високи натоварвания. Предлага разширени възможности за индексирание, включително Full-Text Search и GiST индекси, които могат да ускорят сложни заявки.
- **Съвместимост с Django:** Django има отлична вградена поддръжка за PostgreSQL, което прави интеграцията безпроблемна. Много от специфичните за PostgreSQL функции са достъпни директно през ORM на Django.

2.3 Библиотеки и зависимости

Екосистемата на Python и Django предлага множество библиотеки, които улесняват решаването на конкретни проблеми. В MedJ са използвани няколко ключови библиотеки:

- **django-parler**^[26]: Тази библиотека осигурява лесен начин за добавяне на многоезичност към моделите в Django. В MedJ тя се използва за превод на

съдържание като имена на специалности, типове документи и други номенклатури, което позволява интерфейсът и данните да бъдат визуализирани както на български, така и на английски език.

- **ReportLab^[27] и PyPDF2^[28]:** Комбинацията от тези две библиотеки се използва за генериране и манипулация на PDF документи. ReportLab позволява програмното създаване на PDF файлове от нулата, което се използва за функцията за експорт на отчети. PyPDF2, от своя страна, се използва за задачи като разделяне или сливане на съществуващи PDF файлове, ако е необходимо.
- **django-widget-tweaks^[29]:** Малка, но изключително полезна библиотека, която улеснява добавянето на HTML атрибути (като CSS класове, placeholder текст и др.) към полетата на формите директно в Django шаблоните. Това прави работата с Tailwind CSS и формите на Django много по-гъвкава и чиста.
- **Pillow^[30]:** Библиотека за обработка на изображения, която е необходима за работа с качените от потребителите файлове, например за създаване на миниатюри (thumbnails) или за предварителна обработка преди подаването им към OCR услугата.
- **Psycopg2^[31]:** Това е най-популярният PostgreSQL адаптер за Python, който осигурява връзката между Django приложението и базата данни.

2.4 Инструменти за разработка

Ефективният процес на разработка се подпомага от правилния избор на инструменти:

- **Система за контрол на версиите:** GitHub^[32] е платформата, избрана за хостинг на Git репозиторието на проекта. Използването на Git е стандарт в софтуерната индустрия и позволява лесно проследяване на промените, работа в екип чрез клониране (branching) и сливане (merging) на код, както и връщане към предишни версии при необходимост.
- **Интегрирана среда за разработка (IDE):** Разработчиците по проекта използват PyCharm^[33] или Visual Studio Code^[34] (VS Code). И двете среди предлагат мощни инструменти за писане на Python и JavaScript код, включително дебъгър, статичен анализ на кода, интеграция с Git и множество разширения, които повишават продуктивността.
- **Команден интерпретатор и управление на средата:** За взаимодействие със системата, изпълнение на скриптове и управление на Docker контейнерите се използват **PowerShell** (в Windows среда) и Docker Desktop, който предоставя удобен графичен интерфейс за управление на контейнери, образи и токове (volumes).

Глава 3. Анализ на изискванията към проекта

Успехът на всеки софтуерен продукт зависи не толкова от използваните технологии, колкото от това дали той отговаря на реалните нужди и очаквания на своите потребители. Фазата на анализ на изискванията е критичен етап в жизнения цикъл на софтуерната разработка, който служи като мост между идентифицирания проблем и проектирането на неговото решение. Преди да се пристъпи към дефинирането на архитектурата и техническата реализация на "MedJ", е от ключово значение да се извърши задълбочен и многоаспектен анализ на изискванията. Този процес включва разбиране на проблемите, болките и целите на целевата аудитория, дефиниране на основните функционални и нефункционални изисквания и тяхното приоритизиране, за да се създаде продукт, който е не само функционален, но и полезен, интуитивен и достоен за доверие.

3.1 Методология на проучването

За събиране на първични, емпирични данни относно нуждите, нагласите и проблемите на потенциалните потребители, е проведена онлайн анкета. Този количествен метод е избран, за да се валидират първоначалните хипотези за проблемите с хартиената документация и да се събере информация от по-широк кръг респонденти, което позволява идентифицирането на статистически значими тенденции.

3.1.1 Целева аудитория на проучването:

Въпреки че MedJ е приложение, насочено основно към пациенти (първични потребители), за целите на това първоначално проучване е избрана по-специфична целева аудитория - медицински специалисти (лекари, медицински сестри, стоматолози, фармацевти, психолози и др.). Тази стратегическа селекция е продиктувана от няколко съображения:

- **Експертна гледна точка:**

Медицинските специалисти ежедневно се сблъскват с последствията от лошо управляваната пациентска документация. Техният поглед е професионален и обективен относно неефективността на настоящите процеси.

- **Вторични потребители:**

Те са ключова вторична група потребители, които ще бъдат получатели на информацията, споделяна чрез MedJ. Тяхното одобрение и възприемане на подобен инструмент е критично за неговия успех. Ако

лекарите намират дигитално споделената информация за полезна и лесна за работа, това ще насърчи и пациентите да използват платформата.

•Идентифициране на ключови данни:

Лекарите могат най-точно да посочат коя информация е от най-голямо клинично значение и кои функционалности биха спестили най-много време и усилия по време на преглед.

3.2 Инструмент и разпространение:

Анкетата е създадена с помощта на Google Forms^[35] и разпространена чрез социални мрежи, в частност в професионални групи на медицински специалисти в България. Анонимността на респондентите е гарантирана, за да се насърчи откровеността на отговорите. В рамките на периода на проучването са събрани и анализирани над 80 валидни отговора от специалисти с широк спектър от специалности (обща медицина, ендокринология, кардиология, неврология, дентална медицина, педиатрия и др.) и от различни възрастови групи.

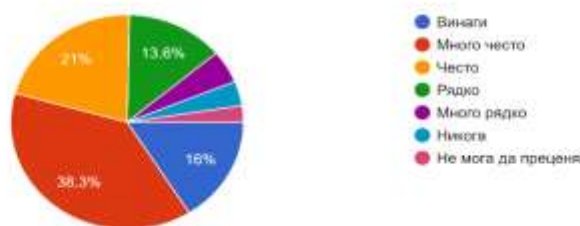
3.3 Анализ на резултатите от анкетата:

Анализът на събраните данни разкри няколко ключови и неоспорими тенденции, които служат като основа за дефиниране на изискванията към проекта:

•Проблемът с хартиените документи е реален и времеемък:

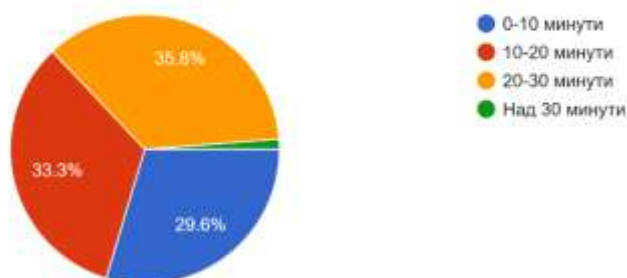
На въпроса "Колко често пациентите ви носят хартиени медицински документи?", 75% от анкетираните отговарят с "Често", "Много често" или "Винаги". Това потвърждава, че физическата документация е преобладаващ стандарт. Относно времето, необходимо за преглед на тези документи, над 69% посочват, че им отнема между 10 и 30 минути - значителна част от времето, отделено за преглед, което би могло да се използва за по-ефективна комуникация с пациента или за по-задълбочен анализ.

3. Колко често пациентите ви носят хартиени медицински документи и резултати от предходни прегледи и изследвания?
81 responses



Фигура 2. Резултати от анкета: Колко често пациентите ви носят хартиени медицински документи и резултати от предходни прегледи и изследвания?

2. Когато пациент носи физически документи (кръвни тестове, рецепти, епикризи), колко време ви отнема да ги прегледате?
81 responses



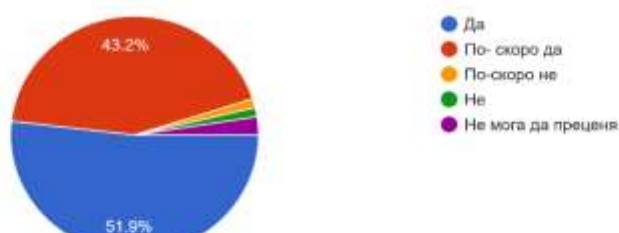
Фигура 3. Резултати от анкета: Когато пациент носи физически документи (кръвни тестове, рецепти, епикризи), колко време ви отнема да ги прегледате?

От резултатите става ясно, че при средна дължина на един преглед 20 минути, 15 от тях са отделени за преглеждане и анализиране на дадената хартиена информация, което изобщо не е продуктивно. На преглеждащия не му остава време за реален разговор с пациента, разчита основно на хартиените носители, които от своя страна зависят от уменията на пациента да води хронологичен запис на всичко.

• Изключително висока степен на одобрение за дигитализация:

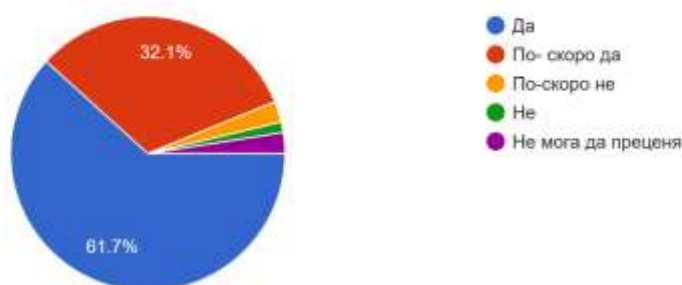
Налице е почти пълно единодушие относно ползите от дигиталните решения. На въпросите "Би ли ви било полезно дигитално структурирано представяне на медицинските данни (графики, обобщения)?" и "Би ли Ви било полезно, ако пациентите ви могат да споделят медицинската си информация чрез линк, QR код...?", над 95% от отговорите са "Да" или "По-скоро да". Това е ясен индикатор, че медицинската общност не само е отворена, но и активно желае и вижда необходимост от инструменти като MedJ, които да модернизират и оптимизират работния процес.

4. Би ли ви било полезно дигитално структурирано представяне на медицинските данни (графики, обобщения, AI-извлечена информация)?
81 responses



Фигура 4. Резултати от анкета: Би ли ви било полезно дигитално структурирано представяне на медицинските данни (графики, обобщения, AI-извлечена информация)?

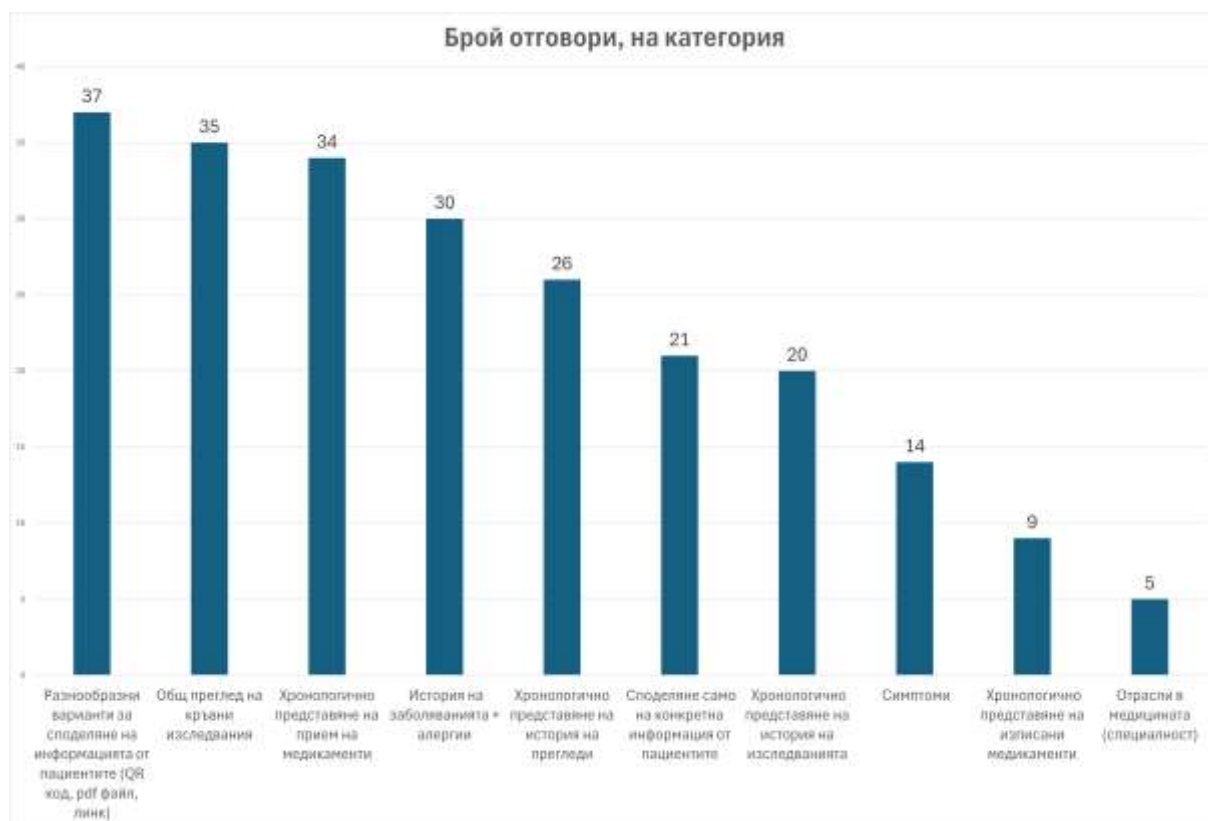
5. Би ли Ви било полезно, ако пациентите ви могат да споделят медицинската си информация чрез линк, QRкод или PDF, вместо да носят хартиени документи?
81 responses



Фигура 5. Резултати от анкета: Би ли Ви било полезно, ако пациентите ви могат да споделят медицинската си информация чрез линк, QRкод или PDF, вместо да носят хартиени документи?

• Идентифициране на най-желаните функционалности:

Резултатите от въпроса за трите най-важни функции дават ясна приоритизация, базирана на реални нужди:



Фигура 6. Резултати от анкета: Най-желани функции

1. Разнообразни варианти за споделяне на информацията е най-често избраната опция. Това подчертава критичната нужда от бърз достъп до фундаментална здравна информация, която е основа за всяко медицинско решение.
2. Общ преглед на кръвни изследвания е втората най-важна функционалност. Лабораторните изследвания са основен диагностичен инструмент и възможността за тяхното хронологично проследяване и визуализация се възприема като огромна полза.
3. Хронологично представяне на прием на медикаменти е на трето място, поради липсата на лесен начин за следене в реално време.
4. История на заболяванията – по същата причина, както е и хронологичното представяне на прием на медикаменти, няма лесен начин да се следят и споделят с медицинските лица.

Тези резултати не просто потвърждават първоначалните хипотези на проекта, но и предоставят солидна, базирана на данни, основа за вземане на решения относно обхвата и приоритетите при разработката на MedJ.

3.4 Психологически аспект (UX - User Experience)

При проектирането на система, която борави с толкова лична и чувствителна информация, техническите аспекти са само половината от уравнението. Потребителското изживяване (UX) и психологическите фактори, които влияят на възприемането на продукта, са от решаващо значение за неговия успех и дългосрочното му приемане от потребителите. UX дизайнът за MedJ се фокусира върху три основни стълба: изграждане на доверие, намаляване на когнитивното натоварване и създаване на усещане за контрол.

•Изграждане на доверие чрез дизайн:

Доверието е валутата, с която работи MedJ. Потребителите трябва да се чувстват абсолютно сигурни, че техните данни са защитени. Това се постига не само чрез силна криптография и спазване на GDPR, но и чрез всеки елемент от потребителския интерфейс.

•Прозрачност и яснота:

При регистрация потребителят трябва да бъде ясно информиран какви данни се събират и как ще се използват. Политиката за поверителност трябва да е лесно достъпна и написана на разбираем език.

- **Усещане за сигурност:**

Професионалният, изчистен и консистентен дизайн сам по себе си внушава надеждност. Липсата на грешки, бързото зареждане на страниците и предвидимото поведение на интерфейса допринасят за усещането, че зад продукта стои компетентен екип.

- **Контрол върху данните:**

Най-важният елемент за изграждане на доверие е усещането за контрол. Потребителят трябва да знае, че само той решава кой, кога и каква част от неговата информация може да види. Функцията за споделяне е проектирана именно с тази философия – временни линкове, ясен срок на валидност и възможност за отмяна на достъпа по всяко време.

3.5 Намаляване на когнитивното натоварване:

Медицинската информация е сложна и често свързана със стрес. Задачата на интерфейса на MedJ е да бъде опростяващ фактор, а не допълнителен източник на объркване.

- **Простота и интуитивност:**

Навигацията в приложението е сведена до няколко основни секции (`Табло`, `История`, `Качване`, `Споделяне`). Процесът по качване на документ е максимално опростен до една "drag-and-drop" зона. Използва се ясна иконография и минималистичен дизайн, за да се избегне визуалното претрупване.

- **Водене на потребителя:**

Системата активно води потребителя през по-сложните процеси. Например, след автоматичната обработка на документ, потребителят не е оставен сам да се чуди какво следва, а е пренасочен към ясен екран за потвърждение с призив за действие ("Прегледай и запази").

- **Достъпност:**

Интерфейсът е проектиран с мисъл за потребители от всички възрастови групи и с различни нива на техническа грамотност. Използват се четливи шрифтове, достатъчно голям размер на интерактивните елементи (бутони, линкове) и добър цветови контраст.

3.6 Създаване на усещане за овластяване (Empowerment):

Основната цел на MedJ е да превърне пациента от пасивен пазител на документи в активен участник в управлението на собственото си здраве.

- **Визуализация на данни:**

Графиките на таблото за управление не са просто красив елемент. Те дават на потребителя възможност да види тенденции в собствените си здравни показатели, което го прави по-информиран и ангажиран.

- **Стъпката за потвърждение:**

Решението AI да не записва данни директно в базата данни, а първо да ги представи на потребителя за одобрение, е ключово. То гарантира точността на данните, но по-важното е, че психологически поставя потребителя в ролята на авторитет, който има последната дума.

- **Централизиран контролен панел:**

Усещането, че цялата ти здравна история е на едно място, подредена и достъпна с няколко клика, само по себе си създава силно чувство за ред и контрол над ситуацията.

3.7 Определяне на основните функционалности

Въз основа на проведеното проучване, анализа на съществуващите решения и дефинираните UX принципи, бяха определени и приоритизирани следните основни функционалности за първоначалната версия на MedJ (Minimum Viable Product - MVP). Този подход позволява бързото предоставяне на стойност на крайния потребител, като се фокусира върху решаването на най-належащите проблеми.

- **Сигурна регистрация и автентикация на потребители:** Стандартен, но критичен модул за създаване на акаунт и защитен вход в системата.
- **Интуитивно качване на документи:** Централна функционалност, която позволява на потребителите лесно да добавят файлове (PDF, JPG, PNG) от всяко устройство.
- **Автоматизиран OCR и AI Pipeline:** Ядрото на интелигентната част на системата, което включва:

3.8 Извличане на текст чрез Google Cloud Vision API

Подаване на текста към OpenAI GPT за структуриране на данни (идентифициране на дата, лекар, специалност) и извличане на ключови показатели (при кръвни изследвания).

Генериране на кратко, лесно за четене обобщение на съдържанието на документа.

Процес на верификация от потребителя: Задължителна стъпка, при която извлечените от AI данни се представят на потребителя в редактируема

форма за преглед, корекция и финално одобрение преди запис в базата данни.

- **Хронологичен преглед с филтриране (History):** Основен екран, който визуализира всички медицински събития като времева линия. Ключов елемент е системата за динамично филтриране по дата, тип документ, специалност и потребителски тагове.
- **Визуализация на данни (Dashboard):** Табло за управление, което представя обобщена информация и интерактивни графики. Първоначалният фокус е върху визуализацията на промяната на лабораторни показатели във времето.
- **Сигурно и контролирано споделяне:** Функционалност за генериране на временен, защитен с токен уеб линк към избрана част от медицинската история. Включва визуализация като QR код за бърз достъп по време на преглед и възможност за задаване на срок на валидност и ръчна деактивация.

3.9 Избор на фокус

Обхватът на медицинската документация е практически необятен. Опитът за създаване на система, която обработва перфектно всички възможни видове документи от самото начало, е рецепта за провал. Затова, следвайки принципите на гъвкавата разработка, е направен стратегически избор да се ограничи фокусът на първоначалната версия на MedJ до двата типа документи, които според проучването и общата практика носят най-голяма стойност и са най-често срещани: кръвни изследвания и епикризи.

• Кръвни изследвания:

Изборът им е мотивиран от няколко фактора:

1. Структурирани данни:

Те съдържат количествени, таблични данни, които са идеални за автоматично извличане и последващ анализ.

2. Висока стойност за визуализация:

Проследяването на динамиката на показатели като глюкоза, холестерол или хемоглобин чрез графики предоставя незабавна и лесно разбираема визуална информация за пациента и лекаря.

3. Ключови за хронични заболявания:

Голяма част от пациентите, които биха имали най-голяма полза от MedJ, са тези с хронични състояния, при които редовният мониторинг на кръвните показатели е от съществено значение.

4. Пряк отговор на проучването:

Това беше една от най-желаните функционалности, посочени от анкетираните медицински специалисти.

•**Епикризи:**

Изборът е мотивиран от следните фактори

1. Обобщават важна информация:

Една епикриза съдържа концентрирана информация за значимо медицинско събитие (хоспитализация, сложна процедура), включително окончателни диагнози, проведени изследвания и препоръки за лечение.

2. Перфектен случай за AI:

Дългият, неструктуриран текст на епикризите е идеална задача за възможностите на големите езикови модели да обобщават и извличат ключови същности (entities).

3. Спестяват време на лекарите:

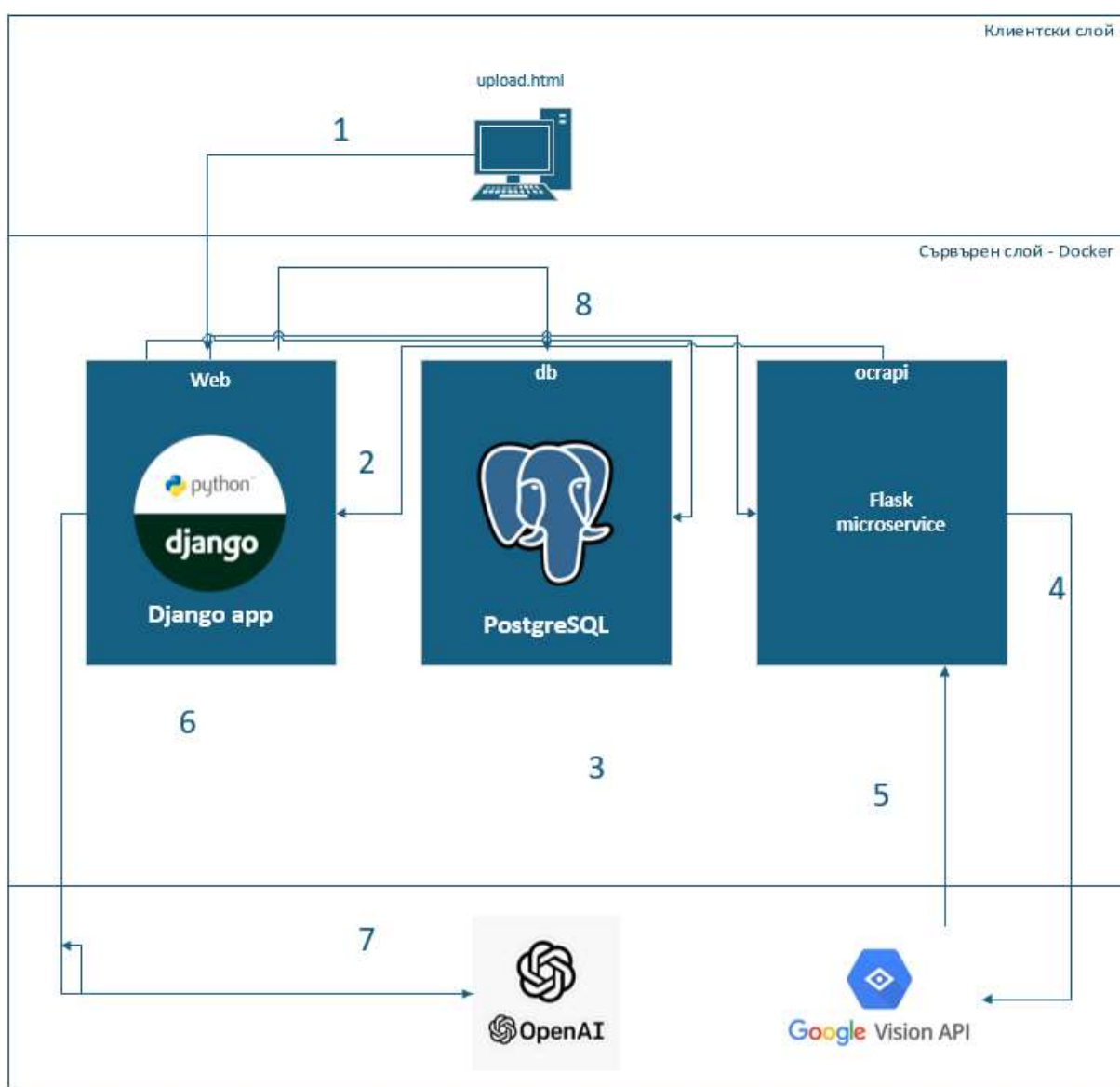
Вместо да чете документ от няколко страници, лекарят може да прегледа генерираното от AI обобщение за секунди, за да се ориентира в случая.

Концентрирайки се върху перфектната обработка на тези два типа документи, MedJ цели да предостави максимална стойност на най-голям сегмент от потребители още в първата си версия. Това създава солидна и тествана основа, върху която в бъдеще лесно могат да бъдат надградени модули за обработка на други видове документация като образни изследвания, рецепти, направления и други.

Глава 4. Архитектура на системата

Добре проектираната архитектура е фундаментална част на всяка стабилна, сигурна и скалируема софтуерна система. Тя не е просто технически план, а стратегическа основа, която дефинира основните структурни компоненти, техните взаимовръзки, каналите за комуникация и ръководните принципи, които управляват тяхното взаимодействие.

Архитектурата на MedJ е щателно разработена, като се придържа към съвременните най-добри практики в уеб разработката. Основните движещи сили зад архитектурните решения са опциите за лесна поддръжка и бъдещи разширения и производителност за осигуряване на гладко потребителско изживяване.



Фигура 7. Архитектура на системата

Изборът на архитектурен модел и технологии е направен след внимателен анализ на целите на проекта: да предостави на потребителите интуитивен и сигурен начин да дигитализират, организират и споделят своята медицинска история. Това изисква система, която може ефективно да обработва качване на файлове, да извършва сложни операции като оптично разпознаване на символи (OCR) и анализ на текст с изкуствен интелект (AI), и същевременно да гарантира поверителността и целостта на данните.

4.1. Общ архитектурен модел

MedJ следва класически многослоен архитектурен модел (N-tier architecture), който разделя приложението на логически и физически разделени компоненти, всеки със строго определена отговорност. Този подход, известен още като "разделяне на отговорностите" (Separation of Concerns), е крайъгълен камък на модерното софтуерно инженерство. Той позволява независимото разработване, тестване, внедряване и мащабиране на отделните части на системата. Например, екипът, работещ по потребителския интерфейс, може да прави промени, без това да засяга бизнес логиката в бекенда, и обратно.

4.1.1 Архитектурата на MedJ е структурирана в четири основни логически слоя:

- **Презентационен слой (Frontend):** Това е клиентската част на приложението, която се изпълнява изцяло в уеб браузъра на крайния потребител. Той е входната врата към системата и е отговорен за визуализацията на данните и за улавянето на потребителските взаимодействия.
- **Приложен слой / Бизнес логика (Backend):** Сърцето на системата, реализирано с уеб рамката Django. Този слой обработва всички HTTP заявки, идващи от потребителския интерфейс, прилага сложната бизнес логика, управлява автентикацията и оторизацията на потребителите и оркестрира комуникацията между базата данни и външните услуги.
- **Слой за персистентност на данните (Database):** Системата за управление на бази данни PostgreSQL, която осигурява надеждно и постоянно (персистентно) съхранение на всички данни на приложението. Това включва потребителски профили, медицински събития, сканирани документи, лабораторни резултати, тагове и др.
- **Слой на външните услуги (External Services):** Това са API (Application Programming Interfaces) на трети страни, които предоставят

специализирана и ресурсоемка функционалност, която не е практично да се разработва вътрешно. В случая на MedJ, това са Google Cloud Vision API за OCR и OpenAI GPT API за анализ и структуриране на медицински текст.

4.1.2 Поток на данните и взаимодействие между компонентите

Типичният процес на взаимодействие между компонентите следва добре установен цикъл "заявка-отговор" (request-response cycle):

1. Потребителско действие:

Потребителят отваря своя уеб браузър и навигира до URL адреса на MedJ. Той взаимодейства с потребителския интерфейс – например, клика върху бутона "Качи документ".

2. HTTP Заявка:

Браузърът конструира и изпраща HTTP заявка (например POST заявка към ендпоинта **/upload/**) към бекенд сървър. Заявката съдържа качените файлове и метаданни.

3. Маршрутизация:

Уеб сървърът приема заявката и я препраща към WSGI сървър, който от своя страна я подава на Django приложението. Основният URL на Django (**medj/urls.py**) анализира пътя и пренасочва заявката към съответния изглед (**view**) в приложението **records**.

4. Обработка в Бекенда:

Изгледът (**view**) в **records/views/upload.py** поема контрола. Той:

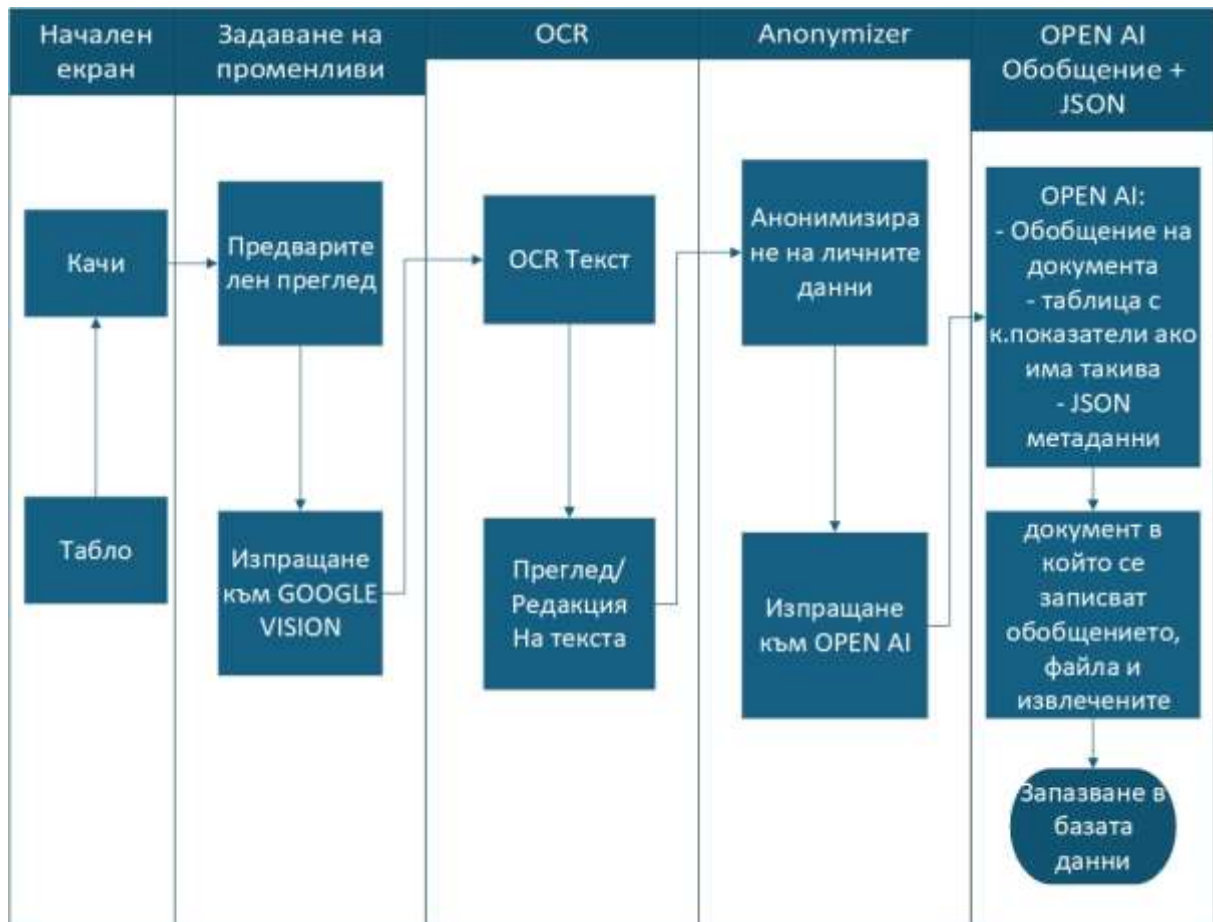
Валидира входящите данни с помощта на Django форма (**MedicalDocumentForm**). Записва временния файл в медийната директория на сървъра.

Извиква вътрешен сервизен модул (**records/ management/ services/ upload_flow.py**), който оркестрира сложната логика. Сервизният модул изпраща файла към Google Cloud Vision API за OCR.

След като получи суровия текст от OCR, модулът го подава на OpenAI GPT API с конкретна инструкция (prompt) да извлече структурирана информация (например лабораторни показатели или ключови термини).

Взаимодейства със слоя за данни, като използва Django ORM, за да създаде нови записи в базата данни – **MedicalEvent**, **MedicalDocument** и др.

5. Визуализация: Браузърът получава отговора и го визуализира на потребителя, с което цикълът завършва.



Фигура 8. От качване до запазване

4.2 Структура на Django проекта

Проектът следва стандартната и силно препоръчителна структура за Django приложения, която насърчава модулността и преизползваемостта на кода. Вместо да се създаде едно монолитно приложение, функционалността е декомпозирана на отделни, самодостатъчни Django "приложения" (apps). Всяко приложение капсулира конкретна бизнес функционалност (например управление на потребители, управление на медицински записи) и има своя собствена вътрешна структура от модели, изгледи, шаблони и URL адреси.

Основни директории и приложения

- **medj/ (Директория на проекта):** Тази директория служи като основен контейнер за конфигурацията на целия проект.
- **records/ (Django приложение):** Това е ядрото на системата. Този подход е избран, тъй като управлението на потребителите е неразривно свързано с

техните медицински данни, и обединяването им в едно приложение намалява сложността и кръстосаните зависимости.

- **models.py:** Дефинира всички модели за базата данни, свързани с медицинската история, чрез Django ORM. Тук са класовете User, MedicalEvent, MedicalDocument, Tag и други, които ще бъдат разгледани в детайли в следващия раздел.
- **views/:** За да се избегне създаването на един огромен views.py файл, логиката за обработка на заявки е разделена на множество файлове по функционален признак:
- **forms.py:** Дефиниция на Django формите, които се използват за валидиране на потребителския вход, като RegistrationForm, MedicalEventForm, PractitionerForm. Формите осигуряват защита срещу често срещани атаки като Cross-Site Scripting (XSS).

4.3 Основни модели и техните връзки

Всички основни модели са дефинирани във файла **records/models.py**. Моделите на данните в MedJ са проектирани да работят в синхрон, за да изградят гъвкава, но същевременно строго структурирана представа за медицинската история на потребителя. Някои модели служат като "номенклатури" (помощни списъци), докато други са "транзакционни" и съхраняват същината на данните.

Ще разгледаме следните ключови модели:

- MedicalEvent (Медицинско събитие) - централният модел
- MedicalDocument (Медицински документ) - моделът за файлове
- Tag (Етикет) - за гъвкаво категоризиране
- Specialty (Специалност) - номенклатурен модел

4.3.1 MedicalEvent (Медицинско събитие)

Това е сърцето на системата, най-важният и централен модел. Той не представлява документ, а по-скоро абстрактен контейнер, който обединява цялата информация, свързана с едно конкретно медицинско взаимодействие във времето.

Предназначение: Да групира всички аспекти на едно събитие – дата, лекар, свързани документи, резултати и потребителски бележки – на едно място.

Основни полета и връзки:

- user: ForeignKey към User
 - **Връзка:** "Many-to-many". Един потребител може да има много медицински събития, но всяко събитие принадлежи само на един потребител.
 - **Значение:** Това е най-критичната връзка за сигурността и целостта на данните. Тя гарантира, че данните на потребителите са стриктно разделени.
- date: DateField
 - **Предназначение:** Съхранява датата, на която се е случило събитието.
 - **Значение:** Това е основното поле, по което се изгражда хронологичният изглед на медицинската история, позволявайки на потребителя да вижда събитията си подредени във времето.
- title: CharField
 - **Предназначение:** Кратко, описателно заглавие, зададено от потребителя, например "Годишен профилактичен преглед", "Изследване на кръвна захар" или "Консултация с кардиолог".
 - **Значение:** Служи за бърза идентификация на събитието в списъците и таблото за управление както и за улеснено филтриране.
- practitioner: ForeignKey към Practitioner
 - **Връзка:** "Many-to-many". Един лекар може да бъде свързан с много събития (на различни пациенти или на един и същ), но едно събитие обикновено се свързва с един конкретен лекар.
 - **Значение:** Позволява проследяване кой специалист е участвал в даденото събитие. Връзката е опционална, защото не всяко събитие е свързано с лекар (напр. домашно измерване на кръвно налягане).
- tags: ManyToManyField към Tag
 - **Връзка:** "Many-to-many". Едно медицинско събитие може да има множество етикети (тагове), и един и същ етикет може да бъде приложен към множество събития.
 - **Значение:** Това е ключовата връзка за гъвкаво филтриране и търсене. Потребителят може да добави тагове като "болничен", "рецепта", "ваксина" и след това лесно да намира всички събития, свързани с тях.

4.3.2 MedicalDocument (Медицински документ)

Този модел представлява **конкретния файл**, който потребителят качва в системата – сканиран PDF, снимка на рецепта, JPG с лабораторни резултати и т.н. Той не съществува самостоятелно, а винаги е **прикачен към MedicalEvent**.

Предназначение: Да съхранява както самия файл, така и метаданните, свързани с него, като например текста, извлечен чрез OCR.

Основни полета и връзки:

- event: ForeignKey към MedicalEvent
- **Връзка:** "Many-to-many". Едно медицинско събитие (MedicalEvent) може да има няколко прикачени документа (например амбулаторен лист и рецепта от същия преглед), но всеки документ принадлежи към точно едно събитие.
- **Значение:** Това е фундаментална връзка, която създава йерархията "Събитие -> Документи". Тя логически групира файловете. Опцията on_delete=models.CASCADE гарантира, че ако събитието-контейнер бъде изтрито, всички свързани с него файлове също ще бъдат изтрети.
- file: FileField
- **Предназначение:** Това не съхранява самия файл в базата данни. Вместо това, то съхранява пътя до мястото, където файлът е физически записан на сървъра (в media директорията).
- **Значение:** Осигурява връзката към реалния качен файл, позволявайки на системата да го достъпва за сваляне, показване или повторна обработка.
- ocr_text: TextField
- **Предназначение:** Голямо текстово поле, в което се записва целият суров текст, разпознат от OCR услугата (Google Cloud Vision).
- **Значение:** Прави съдържанието на документа **търсаемо**. Потребителите могат да търсят по ключови думи (напр. име на медикамент, диагноза), които се съдържат в текста на техните сканирани документи.

4.3.3 Tag (Етикет) и Specialty (Специалност)

Тези два модела изпълняват ролята на номенклатури или "помощни речници". Те не съхраняват същински потребителски данни, а предоставят стандартизирани или дефинирани от потребителя опции за класификация.

4.3.3.1 Tag (Етикет)

Предназначение: Да предостави напълно гъвкав и контролиран от потребителя начин за категоризация. Замества нуждата от твърдо кодирани "категории" или "типове документи".

- Основни полета:
 - name: CharField - Името на етикета (напр. "Кръвни изследвания", "Образна диагностика", "Направление").
- Връзка с MedicalEvent:
 - Например, едно събитие "Преглед при личен лекар" може да има едновременно тагове: "профилактичен", "направление" и "болничен". Същевременно, тагът "болничен" може да се използва и при друго събитие, например "Преглед при УНГ". Това позволява многоизмерно филтриране и организация на данните.

4.3.3.2 Specialty (Специалност)

Предназначение: Да предостави стандартизиран, предварително зададен списък с медицински специалности. За разлика от таговете, този списък не се редактира свободно от потребителите.

- Основни полета:
 - name: CharField - Уникално име на специалността ("Кардиология", "Педиатрия", "Ортопедия" и др).
- Връзка с Practitioner (Лекар):
 - Моделът Practitioner има **ForeignKey** към **Specialty**. Това означава, че когато потребителят добавя нов лекар в своя списък, той избира неговата специалност от този предварително дефиниран списък. Това гарантира консистентност на данните (никой не може да напише "Кардиолог" с правописна грешка) и позволява бъдещи функционалности като търсене на всички събития, свързани с лекари от дадена специалност.

4.4 Seed данни

За да бъде приложението използваемо и полезно веднага след инсталация, е необходимо базата данни да бъде предварително попълнена с някои основни номенклатури. Този процес се нарича "seeding" (засяване) на данните. В MedJ, това се отнася предимно за номенклатурата с медицински специалности (Specialty), тъй като това е стандартна и относително статична информация.

Процесът е реализиран чрез създаването на персонализирана Django management команда: **records/management/commands/seed_initial_data.py**.

Глава 5. Техническа реализация, приложение и бъдещо развитие

Настоящата глава има за цел да навлезе в дълбочина в няколко ключови аспекта, които превръщат архитектурния план в жив, функциониращ продукт.

5.1 Целева аудитория и крайни потребители

Разбирането на крайния потребител е фундаментален процес в софтуерното инженерство, който стои в основата на всеки успешен продукт. Дизайнът, функционалностите и технологичните решения трябва да бъдат пряко следствие от нуждите, очакванията и проблемите на хората, които ще използват системата. За MedJ целевата аудитория може да бъде разделена на две основни, макар и взаимосвързани, групи: пациенти и лекари.

5.1.1 Пациенти (Основна потребителска група)

Пациентите са ядрото на MedJ – те са основните потребители, които генерират, управляват и притежават данните в системата. Профилът на този тип потребител обаче не е хомогенен и обхваща няколко подгрупи:

- **Пациенти с хронични заболявания:** Хора, страдащи от диабет, хипертония, автоимунни или други дългосрочни състояния, които изискват редовно проследяване, чести консултации с различни специалисти и множество лабораторни изследвания. За тях поддържането на организирана медицинска история не е удобство, а необходимост. Те трябва да следят динамиката на определени показатели, да имат бърз достъп до стари епикризи и да могат лесно да предоставят пълна картина на състоянието си на всеки нов лекар.
- **Родители и настойници:** Родители, които управляват медицинската информация на своите деца. Те събират документи от педиатри, специалисти, имунизационни паспорти и резултати от изследвания. Нуждаят се от централизирано място, където да съхраняват всичко и да могат да го споделят лесно при нужда.
- **Хора, грижещи се за възрастни родители:** Често това са технически по-грамотни лица, които помагат на родителите си да се ориентират в здравната система, да съхраняват и споделят техните документи.

5.1.1.1 Проблеми (Pain Points), които MedJ решава за пациентите:

1. **Фрагментация и загуба на информация:** Медицинската история често е разпръсната на десетки хартиени листове, съхранявани в папки,

шкафове или просто изгубени. MedJ предлага единно, сигурно дигитално хранилище.

2. **Недостъпност на данните:** Когато е необходим стар документ, той рядко е под ръка. С MedJ, цялата история е достъпна през всеки уеб браузър, 24/7.

3. **"Мъртви" данни:** Съдържанието на сканираните документи е статично и нетърсаемо. Чрез интеграцията на OCR, MedJ превръща изображенията в текст, който може да бъде търсен, позволявайки на потребителя да намери документ по ключова дума (напр. име на медикамент).

4. **Трудно споделяне:** Предоставянето на медицинска история на нов лекар често означава носене на купчина хартии, което е неефективно и несигурно. Функцията за сигурно споделяне генерира временен, защитен линк, който предоставя елегантен и професионален начин за споделяне на информация.

5. **Липса на хронология:** Трудно е да се проследи развитието на едно състояние във времето, когато документите са разхвърляни. Хронологичният изглед в MedJ (history) подрежда всички събития по дати, създавайки ясна и разбираема времева линия.

5.1.2 **Лекари и медицински специалисти (Вторична потребителска група)**

В текущата си форма, MedJ не е система, предназначена за директна употреба от лекари (т.е. не е болнична информационна система). Лекарите са по-скоро пасивни или вторични потребители, които се явяват получатели на информацията, управлявана от пациентите. Въпреки това, ползите за тях са значителни.

5.1.2.1 **Лекаря-потребител:**

- **Специалисти в частни практики:** Те често работят с пациенти, които идват за второ мнение или за консултация по специфичен проблем и трябва бързо да се запознаят с предисторията им.
- **Лекари в доболничната помощ:** Общопрактикуващи лекари, които координират грижата за пациента и трябва да имат поглед върху консултациите с други специалисти.
- **Спешни медици и лекари в приемни отделения:** В критични ситуации, бързият достъп до информация за хронични заболявания, алергии или приемани медикаменти може да бъде от жизненоважно значение.

5.1.2.2 Проблеми, които MedJ решава за лекарите:

1. **Непълна или неточна анамнеза:** Лекарите често разчитат на паметта на пациента, която може да бъде неточна или непълна. Споделен линк от MedJ предоставя документирана и достоверна информация.
2. **Загуба на време в административна работа:** Време от прегледа се губи в преглеждане на хартиени документи и разпитване за минали събития. С достъп до подредена дигитална история, лекарят може да се фокусира върху настоящия проблем.
3. **Липса на консолидиран поглед:** Пациентът може да е посещавал множество специалисти, но информацията от тях рядко е събрана на едно място. MedJ предоставя тази консолидирана картина.
4. **Трудно разчитане на резултати:** Хартиените распечатки могат да са с лошо качество. Дигиталните копия и особено структурираните лабораторни резултати в MedJ са ясни и лесни за анализ.

Взаимодействието лекар-пациент се подобрява значително, тъй като и двете страни разполагат с един и същ, пълен източник на информация, което води до по-информирани решения и по-високо качество на медицинската грижа.

5.2 Техническа реализация на ключови функционалности

Тук ще разгледаме в детайли как са реализирани два от най-сложните и иновативни процеси в MedJ, като проследим пътя на данните през различните слоеве на приложението.

5.2.1 Процес по качване и интелигентна обработка на документи

Това е основният работен поток в приложението и включва взаимодействие между фронтенд, бекенд и външни AI услуги.

Стъпка 1: Потребителски интерфейс (Frontend)

- Технологии: HTML шаблон (records/templates/main/upload.html), JavaScript (theme/static/js/upload_logic.js), CSS.
- Процес:
 1. Потребителят избира един или няколко файла (изображения или PDF).
 2. Upload_logic.js прихваща събитието за избор на файлове. За всеки избран файл, скриптът динамично създава HTML елементи, които визуализират името на файла, размера му и индикатор за статус.
 3. При натискане на бутона "Качи", данните от формата, включително файловете, датата и заглавието на събитието, се

изпращат към бекенда чрез стандартна POST заявка (multipart/form-data).

Стъпка 2: Обработка в Django изглед (View)

- Файл: records/views/upload.py, функция upload_view.
- Процес:
 1. Изгледът приема POST заявката.
 2. Използва се MedicalEventForm, за да се валидират полетата за дата и заглавие.
 3. Ако формата е валидна, изгледът създава инстанция на MedicalEvent, свързвайки я с текущия потребител (request.user).
 4. За всеки качен файл в request.FILES, изгледът създава инстанция на MedicalDocument, асоциира я със създадения MedicalEvent и записва файла на диска чрез FileField. Статусът на документа първоначално се задава на PROCESSING.
 5. **Ключов момент:** Вместо да извършва тежката обработка директно в изгледа, той делегира тази задача на специализиран сервизен клас. Създава се инстанция на UploadFlowService от **records/management/services/upload_flow.py**, на която се подава новосъздаденият обект MedicalDocument.
 6. Изгледът извиква метода service.run_flow() и пренасочва потребителя към страницата на новосъздаденото събитие.

Стъпка 3: Оркестрация в сервизния слой (Service Layer)

- Файл: records/management/services/upload_flow.py
- Процес: Този клас е отговорен за цялата сложна логика.
 1. OCR (Оптично разпознаване на символи): Методът изпраща HTTP заявка към външния ocrapi микросервиз. Този микросервиз, написан на Flask, получава файла, извиква Google Cloud Vision API (**ocrapi/vision_handler.py**), получава разпознатия текст и го връща обратно на Django приложението. Тази микросервизна архитектура изолира тежките зависимости на Google Cloud библиотеките от основното приложение.
 2. Разпознатият текст се записва в полето ocr_text на обекта MedicalDocument.
 3. AI Анализ (Извличане на структура и тагове):
 - Текстът от OCR се подава на GPTClient (records/management/services/llm/gpt_client.py).
 - Този клиент конструира “prompt” – специфична инструкция за OpenAI GPT модела. Съдържа OCR текста и ясни указания, например: "Анализирай следния медицински текст. Извечи всички лабораторни показатели във формат JSON със следните ключове:

'indicator_name', 'value', 'unit', 'reference_range'. Също така, предложи до 5 релевантни тага за този документ от медицинска гледна точка."

- Изпраща се заявка към OpenAI API.

4. Персистентност на резултатите:

- Полученият JSON с лабораторни резултати се парсва. За всеки резултат се създава инстанция на модела BloodTestResult и се свързва със съответния MedicalEvent.
- Предложените от AI тагове се обработват. За всеки таг системата проверява дали потребителят вече има такъв таг. Ако не, създава се нов (Tag.objects.get_or_create). След това таговете се асоциират със MedicalEvent чрез ManyToManyField.

5. Накрая, статусът на MedicalDocument се променя на COMPLETED.

Стъпка 4: Асинхронна обратна връзка към потребителя

- Файлове: records/views/api_upload.py, theme/static/js /upload_logic.js.
- Процес:
 1. Докато бекендът обработва файла (което може да отнеме секунди), фронтендът не остава пасивен.
 2. upload_logic.js на определен интервал (напр. на всеки 2-3 секунди) изпраща асинхронна GET заявка (AJAX/Fetch) до специален API ендпойнт, дефиниран в api_upload.py (/api/upload-status/<doc_id>).
 3. Този API изглед проверява статуса на документа в базата данни и го връща като JSON отговор: {"status": "PROCESSING"} или {"status": "COMPLETED"}.
 4. Въз основа на отговора, JavaScript кодът обновява статуса на документа в потребителския интерфейс в реално време, без да се презарежда страницата.

5.2.2 . Система за сигурно споделяне на данни

Тази функционалност позволява на пациентите да предоставят временен, само за четене, достъп до избрана част от тяхната медицинска история.

Стъпка 1: Генериране на линк за споделяне

- Технологии: JavaScript (theme/static/js/share_logic.js), Django Views (records/views/share.py, records/views/share_api.py), Django Model (records/models.py).
- Процес:

1. На страницата за споделяне, потребителят вижда списък с всичките си медицински събития.
2. Чрез чекбоксове, той избира кои събития иска да включи в споделянето.
3. При натискане на бутона "Генерирай линк", `share_logic.js` събира ID-тата на избраните събития и срока на валидност на линка (напр. 24 часа, 7 дни).
4. Изпраща се POST AJAX заявка към API ендпойнт в `share_api.py`.
5. Бекенд логика:
 - API изгледът приема списъка с ID-та и срока на валидност.
 - Създава се нова инстанция на модела `ShareLink`.
 - За полето `token` се генерира уникален идентификатор с помощта на `uuid.uuid4()`. Това създава дълъг, криптографски сигурен и практически невъзможен за налучкване низ, напр. `(8c7b5f8a-4c1a-4b0e-8f5a-3e1c9b2d7a0b)`.
 - Изчислява се датата на изтичане (`expires_at`) на базата на текущото време и изборения от потребителя срок.
 - Избраните `MedicalEvent` обекти се прикачат към `ShareLink` инстанцията чрез `ManyToManyField`.
 - API изгледът връща генерирания уникален URL (напр. `https://medj.app/share/8c7b5f8a...`) като JSON отговор.
6. JavaScript кодът получава URL-а и го показва на потребителя, заедно с бутон за лесно копиране.

Стъпка 2: Достъпване на споделения линк

- **Файл:** `records/views/share.py`, функция `share_public_view`.
- **Процес:**
 1. Лекар (или друг получател) отваря получения линк в браузъра си.
 2. URL рутерът на Django (`records/urls.py`) свързва пътя `/share/<uuid:token>/` с `share_public_view`.
 3. Логика на изгледа:
 - Изгледът взема `token`-а от URL-а.
 - Прави заявка към базата данни, за да намери `ShareLink` обект с този токен.
 - Извършва проверки за сигурност:
 - Ако линк с такъв токен не съществува, връща грешка 404 (Not Found).
 - Ако линкът е намерен, проверява дали `expires_at` е в бъдещето. Ако срокът е изтекъл, достъпът се отказва.
 - Ако проверките са успешни, изгледът извлича всички свързани `MedicalEvent` обекти и техните `MedicalDocument`-и.
 - Тези данни се подават на специален, публичен HTML шаблон (`records/templates/subpages/share_public.html`), който ги

визуализира в опростен, само за четене формат. Този изглед не изисква автентикация.

Стъпка 3: Поддръжка и сигурност

- Файл: `records/management/commands/expire_shares.py`.

- Процес:

1. За да се гарантира, че изтеклите линкове не остават активни в системата, е създадена Django management команда.
2. Тази команда може да се настрои да се изпълнява периодично (напр. веднъж на ден) чрез cron job на сървъра.
3. Тя прави заявка към базата данни за всички ShareLink обекти, чиято `expires_at` дата е в миналото, и ги изтрива или маркира като неактивни. Това е важна част от поддръжката и гарантира дългосрочната сигурност на системата.

5.3 Потенциал и перспективи за бъдещо развитие

Настоящата версия на MedJ представлява стабилна основа (Minimum Viable Product), която решава реални проблеми. Нейната архитектура обаче е проектирана с мисъл за бъдещето и позволява множество посоки за разширяване.

5.3.1 Разширения на функционалността

- **Графики и визуализация на данни:** За пациенти с хронични заболявания е изключително важно да следят динамиката на лабораторните си показатели. Да се разработи модул, който взима данните от BloodTestResult и ги визуализира като интерактивни графики. Това ще позволи на потребителите и техните лекари лесно да забелязват тенденции, пикове и спадове във времето.
- **Управление на медикаменти и напомняния:** Дсе добави нова секция за управление на приеманите лекарства. Това ще включва модели за Medication, Prescription и Schedule. Системата ще може да изпраща напомняния (по имейл или чрез push нотификации при наличие на мобилно приложение) за прием на лекарства, което е критично за пациенти с по-сложни терапевтични схеми.
- **Интеграция с носими устройства (Wearables):** Все повече хора използват смарт часовници и фитнес гривни. MedJ да може да се интегрира с платформи като Apple HealthKit или Google Fit, за да импортира автоматично данни за сърдечен ритъм, физическа активност, качество на съня и други показатели. Това ще обогати медицинската история с обективни данни от ежедневието на пациента.

- **Семеен достъп и управление на профили:** Разработване на система за разрешения, която позволява на един потребител (напр. родител) да управлява профила на друг (напр. дете) или да предостави достъп за четене или редакция на свой близък (напр. съпруг/а). Това ще изисква имплементиране на по-сложна логика за оторизация на ниво обекти.

5.3.2 **Технологични и архитектурни подобрения**

- **Преминаване към асинхронна обработка:** В момента обработката на документи (OCR и AI анализ) е синхронна, което означава, че уеб сървърът е зает, докато тя трае. При по-голям брой потребители това ще доведе до проблеми с производителността. Следващата логична стъпка е интеграцията на система за асинхронни задачи като **Celery** с **Redis** като брокер на съобщения. Процесът ще се промени така:
 1. Изгледът за качване приема файла.
 2. Вместо да извиква сервиса директно, той създава задача и я публикува в опашката на Celery.
 3. Изгледът връща моментален отговор на потребителя.
 4. Отделни процеси (Celery workers), които работят на заден фон, взимат задачите от опашката и извършват тежката обработка, без да блокират основното приложение. Това ще направи системата в пъти по-скалируема и отзивчива.
- **Разработване на REST API и мобилно приложение:** За да бъде наистина удобна, системата трябва да има нативно мобилно приложение за iOS и Android. Това налага изграждането на цялостен REST API с помощта на Django REST Framework.

Заклучение

Проектът беше мотивиран от нарастващата и все по-осезаема нужда от дигитализация в сектора на здравеопазването, както и от идентифицираните слабости на съществуващите решения – по-конкретно, липсата на централизирана и изцяло управляема от пациента медицинска история. В епоха, в която дигиталната трансформация засяга всеки аспект от живота, здравеопазването често изостава, оставяйки пациентите в ролята на пасивни носители на фрагментирана, хартиена документация, което води до неефективност, риск от загуба на данни и възпрепятстване на качествената медицинска грижа.

В хода на работата бяха постигнати всички основни цели, заложили в увода на дипломния проект. Разработеното приложение MedJ успешно предоставя на потребителите интуитивна, сигурна и функционално богата платформа за управление на тяхната здравна информация. Крайният продукт съответства на поставените цели, като реализира следните ключови функционалности, които бяха технически имплементирани и подробно описани в предходните глави:

- **Централизирано и сигурно хранилище:** Системата, изградена върху надеждната PostgreSQL база данни и подсигурана с вградените механизми на Django, позволява на потребителите да събират всички свои медицински документи на едно място. Това елиминира риска от загуба и хаоса на хартиения архив, като данните се съхраняват персистентно и защитено.
- **Интелигентна обработка на данни:** Чрез интеграцията на модерни OCR и AI технологии, а именно Google Cloud Vision API и OpenAI GPT, MedJ автоматизира процеса по дигитализация и структуриране на информация от документи като кръвни изследвания и епикризи. Разработеният UploadFlowService, който оркестрира този процес, демонстрира как сложна поредица от асинхронни операции може да бъде управлявана ефективно, за да се спестят време и усилия на потребителите.
- **Визуализация и анализ на данни:** Платформата предлага инструменти за визуализация на лабораторни данни под формата на интерактивни графики, реализирани с помощта на JavaScript библиотеката Chart.js. Тази функционалност, достъпна в таблото за управление (Dashboard), улеснява проследяването на здравни показатели в динамика и превръща сухите данни в лесно разбираема визуална информация.
- **Сигурно и лесно споделяне:** Имплементираният механизъм за споделяне, базиран на модела ShareLink, позволява генерирането на временни, защитени с уникален токен линкове, които могат да бъдат визуализирани и като QR кодове. Този подход, валидиран като изключително необходим от проведената анкета сред медицински

специалисти, решава един от основните проблеми в комуникацията пациент-лекар, правейки процеса бърз, ефективен и сигурен.

Основният принос на настоящия проект се състои в предоставянето на практическо софтуерно решение, което адресира конкретен и значим проблем в сферата на личното здравеопазване. За разлика от институционално-ориентирани системи като националната система "еЗдраве", MedJ поставя пациента в центъра и му дава пълен контрол върху неговата медицинска информация.

Проектът успешно демонстрира как съвременен и разнороден технологичен стек (Python/Django, PostgreSQL, Docker, Tailwind CSS) може да бъде интегриран с модерни облачни AI/OCR услуги за създаването на интелигентно и практическо приложение. Архитектурното решение за изнасяне на OCR процеса в отделен микросървис (oscarpi) е пример за добър инженерен подход, който осигурява скалируемост и отказоустойчивост на системата.

MedJ предоставя инструмент, който овластява пациентите, трансформирайки ги от пасивни притежатели на документи в активни мениджъри на собственото си здраве. Резултатите от проведената анкета недвусмислено показаха, че над 95% от медицинските специалисти виждат огромна полза в дигиталното представяне и споделяне на данни. Спестяването на 10 до 30 минути от времето за преглед, както посочват над 60% от анкетираните, е пряко доказателство за потенциала на MedJ да оптимизира работния процес в здравеопазването и да подобри качеството на медицинската услуга.

Проектът предлага работещ модел за това как персоналните здравни дневници могат да допълнят националните здравни системи, запълвайки празнината в персонализацията и потребителския контрол. MedJ не се стреми да замени "еЗдраве", а да съществува паралелно, давайки възможност на потребителите да обогатят своето дигитално досие с документи от частни прегледи, изследвания в чужбина или исторически хартиени архиви, които иначе биха останали извън обхвата на националната система. Цели се да се улеснят както пациентите в пренасянето и споделянето на тяхната важна за здравето информация, но и също така да се помогне максимално на вече така или иначе претоварена здравна система.

Въпреки постигнатите резултати, е важно да се отбележат и някои ограничения на настоящата реализация, които очертават пътя за бъдещо развитие. Проектът, в ролята си на минимално работещ продукт (MVP), съзнателно се фокусира върху обработката само на кръвни изследвания и епикризи. Освен това, към момента MedJ съществува само като уеб приложение, което може да бъде неудобство за някои потребители, свикнали с мобилни приложения.

Разработената система MedJ представлява стабилна основа, която има значителен потенциал за бъдещо развитие и разширяване.

Перспективите за надграждане са многобройни и вълнуващи бъдещи усилия могат да се насочат към интеграция с лаборатории и болници за автоматизиран обмен на данни, което би елиминирало напълно нуждата от ръчно качване, функционалността може да бъде обогатена с модули за проследяване на прием на медикаменти, система за автоматични известия и напомняния.

На първо място винаги са крайните потребители а в този случай държавната система има видими пропуски. Липсата на автономност и право на решение кой и кога да има достъп до личните данни на човек е от съществено значение. Целта на този проект е да бъде както удобство за медицинските лица, така и да върне контрола върху здравето обратно на пациента.

В заключение, проектът MedJ успешно демонстрира как съвременните софтуерни технологии могат да бъдат приложени за създаването на иновативни и социално значими решения в здравеопазването. Чрез поставянето на пациента в центъра на процеса по управление на здравната информация, MedJ не само решава конкретни практически проблеми, но и допринася за една по-достъпна, ефективна и ориентирана към индивидуалните нужди медицинска грижа. Постигнатите резултати и очертаните перспективи за развитие утвърждават потенциала на проекта да се превърне в ценен инструмент в ръцете на всеки, който се стреми към по-осъзнато и проактивно отношение към собственото си здраве.

Библиография

1. skener.news: Отчетоха пълна дигитализация в здравеопазването, <https://skener.news/2024/06/14/%d0%be%d1%82%d1%87%d0%b5%d1%82%d0%be%d1%85%d0%b0-%d0%bf%d1%8a%d0%bb%d0%bd%d0%b0-%d0%b4%d0%b8%d0%b3%d0%b8%d1%82%d0%b0%d0%bb%d0%b8%d0%b7%d0%b0%d1%86%d0%b8%d1%8f-%d0%b2-%d0%b7%d0%b4%d1%80%d0%b0%d0%b2/>, (2024)
2. НАРЕДБА № 8 от 3.11.2016 г. за профилактичните прегледи и диспансеризацията
3. Новини - Министерство на електронното управление, https://egov.government.bg/wps/portal/ministry-meu/press-center/news/news251023!/ut/p/z0/fYwxD4IwFIR_iwMjeQ-IiGNjItiEhLG-xVRSoWoKtI3ov7eos8vl7vLdAYEAMvKhO-n1Y0Q95CPlp6YpWVUWiEld5pgzzvfbgh2qbAMc6D8QHlJb7-oOaJS-j7W5DCCMml26TjDNFkBfp4kYUDsYr54exMcYH6HTXkU4WuVc3IZG2QiX7Vd_D-ONzq-Zrd50JmLR/
4. Национална здравноинформационна система: Обща информация, <https://www.medentic.app/bg/blog/national-health-information-system-bulgaria-overview>
5. АД, И. обслужване: еЗдраве • МЗ, <https://www.mh.government.bg/bg/informaciya-za-grazhdani/eZdrave>
6. Apple Health vs Google Fit: What is the difference?, <https://versus.com/en/apple-health-vs-google-fit>
7. Oza, H.: Google Fit vs Apple Health: Digital Health Industry | Hyperlink InfoSystem, <https://www.hyperlinkinfosystem.ca/blog/google-fit-vs-apple-health-in-digital-health-industry>
8. OCR Accuracy Showdown: Tesseract vs. Google Cloud Vision vs. ABBYY, <https://eureka.patsnap.com/article/ocr-accuracy-showdown-tesseract-vs-google-cloud-vision-vs-abbyy>
9. OCR Solutions Compared: Artificio vs. Google vs. Amazon, <https://artificio.ai/blog/comparing-modern-ocr-solutions-a-comprehensive-analysis>
10. Isabelle Gribomont, “OCR with Google Vision API and Tesseract,” Programming Historian 12 (2023), <https://doi.org/10.4643>
11. Health | European Data Protection Supervisor, https://www.edps.europa.eu/data-protection/our-work/subjects/health_en
12. Data Protection Guide Bulgaria, https://multilaw.com/Multilaw/Multilaw/Data_Protection_Laws_Guide/DataProtection_Guide_Bulgaria.aspx

13. Python Release Python 3.12.0,
<https://www.python.org/downloads/release/python-3120/>
14. Django documentation | Django documentation,
<https://docs.djangoproject.com/en/5.0/>
15. HTML5,
<https://bg.wikipedia.org/w/index.php?title=HTML5&oldid=12000562>, (2023)
16. Tailwind CLI - Installation, <https://tailwindcss.com/docs/installation/tailwind-cli>
17. JavaScript,
<https://en.wikipedia.org/w/index.php?title=JavaScript&oldid=1306812934>,
(2025)
18. Themesberg: Flowbite - Tailwind CSS component library,
<https://flowbite.com/docs/getting-started/introduction/>
19. Docker: lightweight Linux containers for consistent development and deployment,
https://www.researchgate.net/publication/261960832_Docker_lightweight_Linux_containers_for_consistent_development_and_deployment
20. Docker Compose, <https://docs.docker.com/compose/>
21. What Is PostgreSQL? Databases Explained,
<https://cloud.google.com/discover/what-is-postgresql>
22. Detect text in images | Cloud Vision API,
<https://cloud.google.com/vision/docs/ocr>
23. Model - OpenAI API, <https://platform.openai.com>
24. What is PostgreSQL?, <https://aws.amazon.com/rds/postgresql/what-is-postgresql/>
25. JSON, <https://www.json.org/json-en.html>
26. django-parler/django-parler: Easily translate “cheese omelet” into “omelette au fromage”., <https://github.com/django-parler/django-parler>
27. ReportLab PDF Generation User Guide,
<https://www.reportlab.com/docs/reportlab-userguide.pdf>
28. PyPDF2: A pure-python PDF library capable of splitting, merging, cropping, and transforming PDF files
29. django-widget-tweaks: Tweak the form field rendering in templates, not in python-level form definitions., <https://github.com/jazzband/django-widget-tweaks>
30. pillow: Python Imaging Library (Fork), <https://python-pillow.github.io>
31. psycopg2: psycopg2 - Python-PostgreSQL Database Adapter,
<https://psycopg.org/>
32. GitHub.com Help Documentation, <https://docs-internal.github.com/en>
33. PyCharm: The only Python IDE you need,
<https://www.jetbrains.com/pycharm/>
34. Visual Studio Code - Code Editing. Redefined, <https://code.visualstudio.com/>

35. Overview | Google Forms | Google for Developers, <https://developers.google.com/workspace/forms/api/guides>
36. Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*, 33, 1877-1901.
37. Chacon, S., & Straub, B. (2014). *Pro Git*. Apress.
38. European Parliament and Council of the European Union. (2016). Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation). *Official Journal of the European Union*, L 119/1.
39. Fowler, M. (2021). *The Great Server-Side/Client-Side Divide*. MartinFowler.com.
40. Grigorov, A. (2022). *Digitalization of Healthcare in Bulgaria: Challenges and Perspectives*. Sofia University "St. Kliment Ohridski".
41. Ramson, S. J., & R. A. (2018). A Study on Optical Character Recognition using Tesseract. *International Journal of Innovative Science and Research Technology*, Volume 3, Issue 6.
42. Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media.
43. Beck, K. (2000). *Extreme Programming Explained: Embrace Change*. Addison-Wesley Professional.
44. Cairo, A. (2016). *The Truthful Art: Data, Charts, and Maps for Communication*. New Riders.
45. Cushman, W. H., & Rosenberg, D. J. (1991). *Human Factors in Product Design*. Elsevier Science.
46. Dreyer, D. (2021). *Two Scoops of Django 3.x: Best Practices for the Django Web Framework*. Two Scoops Press
47. Krug, S. (2014). *Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability*. New Riders.
48. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press
49. Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*.
50. Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.
51. Shneiderman, B., Plaisant, C., Cohen, M., Jacobs, S., & Elmqvist, N. (2017). *Designing the User Interface: Strategies for Effective Human-Computer Interaction* (6th ed.). Pearson.
52. The Center for Medicare & Medicaid Services. (2023). *Security 101 for Covered Entities*. U.S. Department of Health & Human Services.

Приложения:

1. records/models.py

```
class MedicalCategory(TranslatableModel):  & 444
    translations = TranslatedFields(
        name=models.CharField(max_length=255, unique=True),
        description=models.TextField(blank=True, null=True),
    )
    slug = models.SlugField(max_length=255, unique=True, blank=True, null=True)
    is_active = models.BooleanField(default=True)
    order = models.PositiveIntegerField(default=0)

    def save(self, *args, **kwargs):  & 444
        if not self.slug:
            name = self.safe_translation_getter(field="name", any_language=True) or ""
            self.slug = slugify(name)[:255] or None
        super().save(*args, **kwargs)

    def __str__(self):  & 444
        return self.safe_translation_getter(field="name", any_language=True) or "Category"

class MedicalSpecialty(TranslatableModel):  & 444
    translations = TranslatedFields(
        name=models.CharField(max_length=255, unique=True),
        description=models.TextField(blank=True, null=True),
    )
    slug = models.SlugField(max_length=255, unique=True, blank=True, null=True)
    is_active = models.BooleanField(default=True)
    order = models.PositiveIntegerField(default=0)

    def save(self, *args, **kwargs):  & 444
        if not self.slug:
            name = self.safe_translation_getter(field="name", any_language=True) or ""
            self.slug = slugify(name)[:255] or None
        super().save(*args, **kwargs)

    def __str__(self):  & 444
        return self.safe_translation_getter(field="name", any_language=True) or "Specialty"
```

2.records/admin.py

```
@admin.register(PatientProfile) & 444
class PatientProfileAdmin(admin.ModelAdmin):
    list_display = ("user", "share_enabled", "share_token")
    search_fields = ("user__username", "user__email")

@admin.register(MedicalCategory) & 444
class MedicalCategoryAdmin(admin.ModelAdmin):
    list_display = ("slug", "is_active", "order")
    list_filter = ("is_active",)
    search_fields = ("translations__name",)

@admin.register(MedicalSpecialty) & 444
class MedicalSpecialtyAdmin(admin.ModelAdmin):
    list_display = ("slug", "is_active", "order")
    list_filter = ("is_active",)
    search_fields = ("translations__name",)

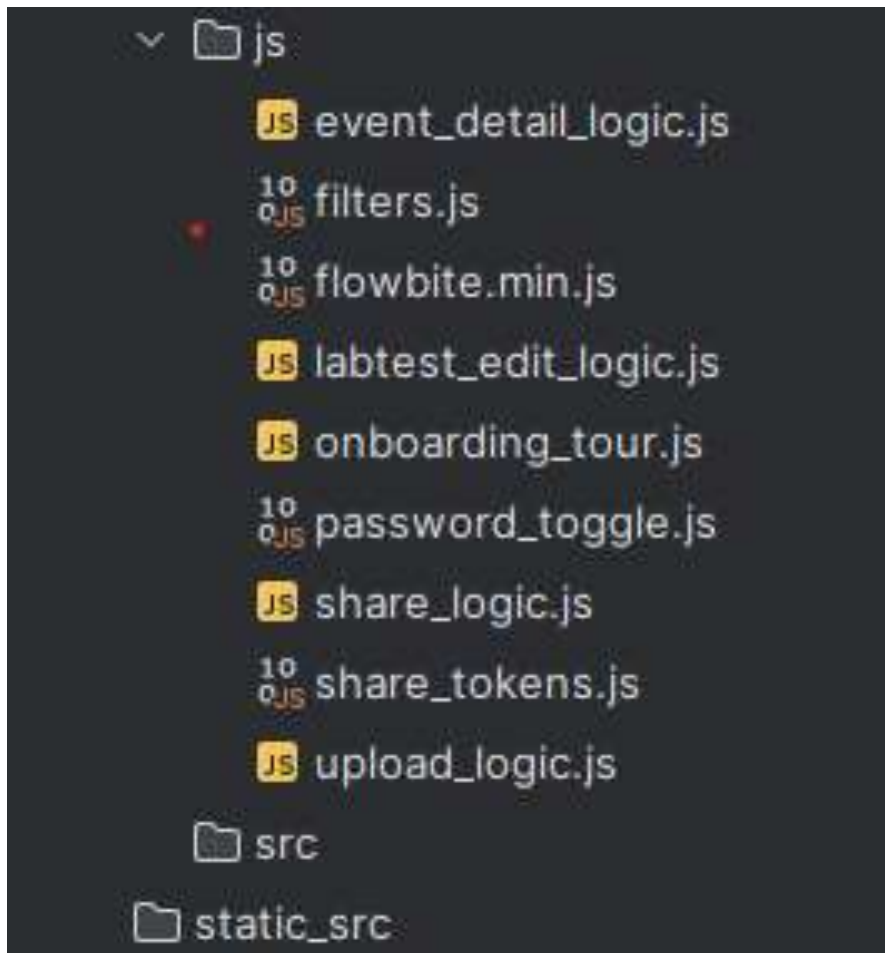
@admin.register(DocumentType) & 444
class DocumentTypeAdmin(admin.ModelAdmin):
    list_display = ("slug", "is_active", "order")
    list_filter = ("is_active",)
    search_fields = ("translations__name",)

@admin.register(Tag) & 444
class TagAdmin(admin.ModelAdmin):
    list_display = ("slug", "kind", "is_active")
    list_filter = ("kind", "is_active")
    search_fields = ("slug", "translations__name")

@admin.register(MedicalEvent) & 444
class MedicalEventAdmin(admin.ModelAdmin):
    list_display = ("id", "patient", "owner", "specialty", "category", "doc_type", "event_date", "created_at")
    list_filter = ("specialty", "category", "doc_type", "event_date")
    search_fields = ("patient__user__username",)

@admin.register(EventTag) & 444
class EventTagAdmin(admin.ModelAdmin):
    list_display = ("event", "tag")
    list_filter = ("tag__kind",)
```

3.theme/static/js/



4.docker/web/Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
ENV PYTHONUNBUFFERED=1
ENV PYTHONOPTIMIZE=1
RUN apt-get update && apt-get install -y build-essential libpq-dev gettext curl postgresql-client && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
RUN curl -L -o /usr/local/bin/tailwindcss https://github.com/tailwindlabs/tailwindcss/releases/download/v4.0.0/tailwindcss-linux-x64 && chmod +x /usr/local/bin/tailwindcss
COPY . .
ENTRYPOINT ["sh", "docker/web/entrypoint.dev.sh"]
```

5.requirements.txt

```
Django
psycpg2-binary
Pillow
whitenoise
django-environ
django-widget-tweaks
django-parler
requests
python-dateutil
pytz
qrcode[pil]
openai
python-dotenv
google-cloud-vision
google-auth
weasyprint
pydyf
tinycss2
cssselect2
reportlab
PyPDF2
```

6.docker/ocrapi/Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
RUN apt-get update && apt-get install -y build-essential tesseract-ocr libtesseract-dev && rm -rf /var/lib/apt/lists/*
COPY ocrapi/requirements-ocr.txt /tmp/requirements-ocr.txt
RUN pip install --no-cache-dir -r /tmp/requirements-ocr.txt
COPY . .
CMD ["python", "-n", "flask", "run", "--host=0.0.0.0", "--port=5000"]
```

7.ocrapi/requirements-ocr.txt

```
Flask==3.0.3  
gunicorn==21.2.0  
requests==2.32.3  
Pillow==10.4.0  
pytesseract==0.3.10  
pymupdf==1.24.9
```


8.records/views/api_upload.py

```
@login_required 1 usage 8 444
@require_http_methods(["POST"])
def upload_analyze(request):
    try:
        if request.content_type and "application/json" in request.content_type:
            body = json.loads(request.body or "{}")
            txt = (body.get("text") or "").strip()
            specialty_id = body.get("specialty_id")
        else:
            txt = (request.POST.get("text") or "").strip()
            specialty_id = request.POST.get("specialty_id") or request.POST.get("specialty")
    except Exception:
        txt, specialty_id = "", None
    if not txt:
        return HttpResponseBadRequest("No text")
    specialty_name = _id_to_name(MedicalSpecialty, specialty_id)
    clean = _anonymize(txt)
    api_key = os.getenv("OPENAI_API_KEY", "").strip()
    model = os.getenv("OPENAI_MODEL", "gpt-4o-mini")
    if api_key:
        try:
            from openai import OpenAI
            client = OpenAI(api_key=api_key)
            sys = _build_system_prompt()
            resp = client.chat.completions.create(
                model=model,
                response_format={"type": "json_object"},
                max_tokens=1200,
                messages=[{"role": "system", "content": sys}, {"role": "user", "content": clean}]
            )
            content = resp.choices[0].message.content or "{}"
            data = json.loads(content)
            summary = (data.get("summary") or "").strip()
            return JsonResponse({"summary": summary, "data": data})
        except Exception:
            pass
    data = _fallback_extract(clean, specialty_name)
    summary = data.get("summary", "")
    return JsonResponse({"summary": summary, "data": data})
```

9.Промт

```
def _build_system_prompt(): 1 usage  444
    return (
        "Върни САМО JSON в UTF-8 без обяснения и без markdown. "
        "{"
        "summary":"","
        "event_date":"","
        "detected_specialty":"","
        "suggested_tags":[],
        "blood_test_results":[{"indicator_name":"","value":"","unit":"","reference_range":""}],
        "diagnosis":"","
        "treatment_plan":"","
        "doctors":[]
        "}"
        " Ако липсва информация за поле, остави празно или празен списък."
    )
```

10.records/views/events.py

```
@login_required 2 usage  444
def events_by_specialty(request: HttpRequest) -> JsonResponse:
    patient = require_patient_profile(request.user)
    spec_val = request.GET.get('specialty_id') or request.GET.get('specialty')
    try:
        spec_id = int(spec_val)
    except (TypeError, ValueError):
        return JsonResponse({"results": []})
    qs = MedicalEvent.objects.filter(patient=patient, specialty_id=spec_id).order_by( "field_names: "-event_date", "-id")
    results = [
        {"id": e.id, "date": e.event_date.strftime("%Y-%m-%d") if e.event_date else "", "summary": e.summary or ""}
        for e in qs
    ]
    return JsonResponse({"results": results})
```